



Technische Universität Berlin
School IV - Electrical Engineering and Computer Science
Intelligent Multi-Robot Coordination Lab

Master Thesis

Learning Multirotor Vision-to-Action Control Policies via Imitation Learning

presented by

Max Klemens

Matr.-Nr.: 486142

M.Sc. Computer Engineering

Date of submission: July 8, 2025

Supervisors:
Akmaral Moldagalieva
Prof. Dr. Wolfgang Hönig

Examiners:
Prof. Dr. Wolfgang Hönig
Prof. Dr. Marc Toussaint

Declaration

English

I hereby declare that I have composed this thesis independently, without the assistance of third parties, and solely using the sources and tools listed. All passages taken from these sources and tools have been duly identified as such.

In cases where generative AI tools were employed, I have specified the product names, manufacturers, the exact software versions used, and their respective purposes. I assume full responsibility for the selection, incorporation, and all outcomes of the AI-generated content utilized.

In the development of this thesis, I employed OpenAI ChatGPT (model o4) for code generation and textual drafting. Its capabilities regarding programming tasks and argumentative clarity were utilized throughout this thesis. Additionally, I leveraged Grammarly to review and enhance writing mechanics, ensuring correctness in grammar and overall readability.

German

Hiermit versichere ich, dass ich die vorliegende Arbeit eigenständig ohne Hilfe Dritter und ausschließlich unter Verwendung der aufgeführten Quellen und Hilfsmittel angefertigt habe. Alle Stellen die den benutzten Quellen und Hilfsmitteln unverändert oder sinngemäß entnommen sind, habe ich als solche kenntlich gemacht.

Sofern generative KI-Tools verwendet wurden, habe ich Produktnamen, Hersteller, die jeweils verwendete Softwareversion und die jeweiligen Einsatzzwecke (z.B. sprachliche Überprüfung und Verbesserung der Texte, systematische Recherche) benannt. Ich verantworte die Auswahl, die Übernahme und sämtliche Ergebnisse des von mir verwendeten KI-generierten Outputs vollumfänglich selbst.

Die Satzung zur Sicherung guter wissenschaftlicher Praxis an der TU Berlin vom 15. Februar 2023 habe ich zur Kenntnis genommen. Ich erkläre weiterhin, dass ich die Arbeit in gleicher oder ähnlicher Form noch keiner anderen Prüfungsbehörde vorgelegt habe.

Berlin,
July 8, 2025



Max Klemens

Abstract

This thesis investigates autonomous quadrotor navigation by developing and evaluating two vision-to-action drone racing policies trained solely in simulation via imitation learning. Of the two developed policies, one was trained end-to-end to predict actions directly on raw images, while the other network contains a perception submodule to pre-process image features. The conceived strategies learn from an expert motion planner, iterative discontinuity-bounded A^* (iDbA *).

In simulation, our end-to-end trained policy achieved lap times and path lengths undercutting those of a human expert, while our developed perception-based policy showed promising results regarding race gate recognition but failed to navigate the complete race track.

However, neither approach transferred successfully to a physical platform. The end-to-end strategy overfitted to the simulation environment, and the perception-driven network failed when racing gates exited the camera’s field of view. From these findings, we identify two primary challenges for sim-to-real transfer in learned vision-based autonomy. First, visual overfitting during training in simulation, and second, loss of state awareness.

By evaluating our end-to-end and perception-driven approaches, we provide insights for the design of future autonomous UAV racing systems.

Deutsche Kurzzusammenfassung

Diese Abschlussarbeit untersucht die autonome Navigation von Quadrotoren, indem zwei bildbasierte künstliche neuronale Netze in Simulation mittels Imitationslernen trainiert und evaluiert werden. Während einer der beiden Agenten Ende-zu-Ende trainiert wurde, d.h. Steuerungsbefehle auf Basis roher RGB-Bilder vorhersagt, enthält das zweite entwickelte neuronale Netz ein Bildvorverarbeitungsmodul zum extrahieren relevanter Bildinformation. Beide Strategien lernen von dem Experten-Bewegungsplaner iterative discontinuity-bounded A* (iDbA*).

In der Simulation erzielte unsere Ende-zu-Ende trainierte Strategie Rundenzeiten und Weglängen, welche die eines menschlichen Piloten unterboten.

Keiner der implementierten Ansätze ließ sich jedoch erfolgreich in der realen Welt umsetzen. Die Ende-zu-Ende trainierte Methode scheiterte aufgrund von Überanpassung an die Simulationsumgebung, und unser zweites, wahrnehmungsgetriebenes Netzwerk scheiterte, wenn die zu passierenden Tore aus dem Kamerasichtfeld verschwanden. Anhand dieser Ergebnisse leiten wir zwei zentrale Herausforderungen für den Transfer von Simulation zu physischer Hardware von erlernten, bildbasierten Autonomiesystemen ab. Erstens, das visuelle Overfitting zur Simulationsumgebung und zweitens, der Informationsverlust über den Zustand des Quadrotors.

Durch den Vergleich unserer implementierten Ansätze liefert diese Arbeit wertvolle Erkenntnisse für die Entwicklung zukünftiger autonomer UAV-Rennsysteme.

Contents

1	Introduction	1
1.1	Motivation	2
1.2	Contribution	3
1.3	Thesis Overview	4
2	Background	5
2.1	Drone Racing	5
2.2	Quadrotor Dynamics	6
2.2.1	Full Dynamics Model	6
2.2.2	Differential Flatness	8
2.2.3	Double Integrator Model	9
2.3	Quadrotor Control	10
2.3.1	Nonlinear Controller Derivation	10
2.3.2	PID Control	12
2.3.3	Model Predictive Control	13
2.3.4	Reinforcement Learning Control	13
2.4	Motion Planning	14
2.4.1	iDbA* Algorithm	15
2.4.2	Rapidly-Exploring Random Trees	16
2.5	Imitation Learning	17
2.5.1	Dataset Aggregation	17
2.5.2	Generative Adversarial Imitation Learning	19
3	Related Work	20
3.1	Autonomous Drone Racing	20
3.1.1	State of the Art	20
3.1.2	Perception	23
3.1.3	Planning	25
3.1.4	Control	26
3.2	Discussion	27
4	Learning a Vision-to-Action Policy for Drone Racing	29
4.1	System Setup	29
4.1.1	Hardware Setup	29
4.1.2	Software Setup	31
4.2	Approach	31
4.2.1	Perception, Planning, and Control	32
4.2.2	Learned Perception	32
4.2.3	Learned Planning	35

4.2.4	Implementation	36
5	Experiments	40
5.1	Developmental Iterations and Failures	40
5.1.1	End-to-End Policy	41
5.1.2	Pencil-Filter Image Processing	42
5.1.3	Gate Direction Perception	43
5.2	Experiment Metrics and Evaluation Criteria	44
5.3	Evaluation	45
5.3.1	Perception Policy	46
5.3.2	Human Piloted Flight	47
5.3.3	Autonomous Drone Racing Policy	48
5.3.4	Quantitative Evaluation	53
6	Discussion and Conclusion	57
6.1	Implications and Limitations	57
6.2	Future Work	58
6.3	Conclusion	59
	Bibliography	60

List of Figures

1.1	FPV Drone Racing	2
2.1	Definition of Quadrotor Frames and Axes Regarding Dynamics Model . . .	6
2.2	iDbA* Motion Planning	16
3.1	Conventional Autonomy Pipeline	20
3.2	Visual Inertial Odometry	23
3.3	Learned Perception by UZH and MavLab	24
4.1	Hardware Setup Quadrotor Communication	30
4.2	Hardware Setup Flight Space with Race Track	31
4.3	Comparison of Synthetic and Camera Image	32
4.4	Implemented Autonomy Pipeline	32
4.5	Policy Architecture	33
4.6	Image Labeling	35
5.1	Pencilfilter in Simulation and Real-World	42
5.2	Discarded Policy Architectures	44
5.3	Perception Policy Training	46
5.4	Human Pilot on Hardware	47
5.5	iDbA* in Simulation	48
5.6	End-to-End Trained Policy in Simulation	49
5.7	End-to-End Trained Policy on Hardware	50
5.8	Perception-Based Policy in Simulation	51
5.9	Loss of State Awareness by Gate Exiting Camera Frame	52
5.10	Perception-Based Policy on Hardware	52
5.11	Rendering of Race Track and Recorded Trajectories	53
5.12	Comparison of Trajectory's Path Lengths	54
5.13	Human Pilot Lap Times	55

1. Introduction

The field of autonomous unmanned aerial vehicles (UAVs) has seen groundbreaking advances over the past decade [1]. These aerial systems are increasingly prominent across diverse domains, ranging from visual mapping [2], surveillance [3], and infrastructure inspection [4], to logistics and package delivery [5]. System properties such as agility, precision, and cost-efficiency make them suitable for applications in which traditional aerial vehicles are impractical.

Similarly, artificial intelligence (AI) has progressed at a fast pace, particularly driven by breakthroughs in vision [6], imitation learning [7], or reinforcement learning [8]. Selected AI systems have been benchmarked in competitive settings against human experts, highlighting their competence in tasks of increasing complexity. Landmarks such as IBM’s Deep Blue defeating world chess champion Garry Kasparov in 1997 [9] and AlphaGo’s 2017 victory over Go champion Ke Jie [10] show AI’s capabilities and their superiority over human competitors in certain domains. However, such milestones occurred within fully observable and discrete domains.

In contrast, robotics applications introduce a higher level of complexity. Robotic agents operate in continuous, high-dimensional, and partially observable environments [11]. Unlike games or simulations, robots must operate in the real world, which is difficult to model and predict in. This challenge is embodied in autonomous UAV racing, among other robot learning applications. Here, drones must navigate 3D tracks at high speeds, relying on onboard sensing while being constrained by size, weight, and power (SWaP) limitations [12]. Although human drone pilots require extensive training to develop the navigation capabilities necessary for success in international competitions, autonomous drone racing aims to surpass human performance.

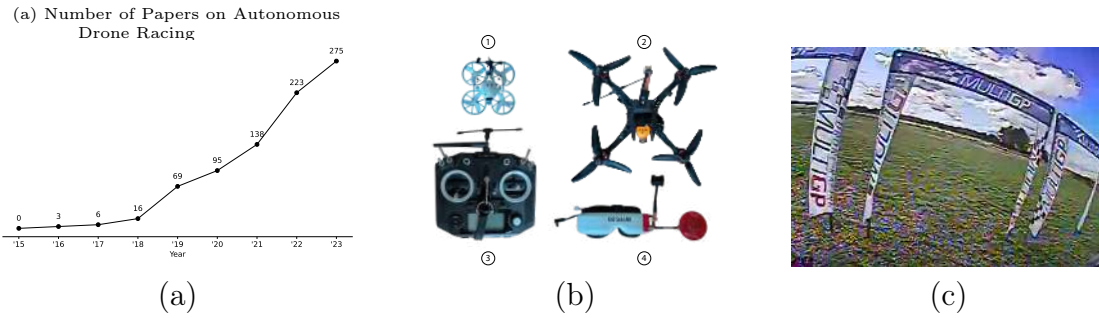


Figure 1.1: The field of drone racing is rapidly growing in terms of an e-sport [13] as well as its autonomous counterpart in Academia. (a) Plot illustrating the number of papers published on autonomous drone racing over the years [14]. The domain has gained high popularity in less than ten years after its initial academic consideration. (b) Image depicting the essential hardware components for conventional drone racing. (1) "Tiny Whoop", a quadrotor, which is one inch in diameter and often deployed in indoor drone races. The low weight restricts its speed and flight characteristics. (2) Five inch first-person view (FPV) racing drone, with high thrust-to-weight ratio. Due to its balance between speed and maneuverability, this type of quadrotor is featured in most drone races. (3) FrSky Taranis QX7 Radio, which transmits the control signal to the quadrotor. Pilots can command inputs via two joysticks representing angular rates and thrust. (4) FatShark HD0 FPV-goggles, which receive and display video from the quadrotor's point of view. (c) A single frame extracted from an FPV feed flying through racing gates.

1.1 Motivation

Historical records trace racing events as far back as 3000 BC in ancient Egypt, where the Pharaoh's physical abilities were tested in ceremonial footraces [15]. Over time, racing has evolved from human runners to chariots, automobiles, airplanes, and recently, drones [13]. Despite these transformations, the foundational purpose of racing remains unchanged.

Racing, as a sport and as a scientific challenge, catalyzes technological innovation. Motor-sports such as Formula 1, IndyCar, and the World Rally Championship have influenced research in vehicle dynamics, control systems, and safety mechanisms. These pushing environments drive innovations that eventually progress commercial automotive technologies [13]. The competitive goal of minimizing lap times fosters the development of vehicles at the edge of physical performance. This is reflected in a statement by Betz et al., capturing the reasons that make racing a driving force in research:

"With a vehicle that moves at the dynamic limits of handling, that reaches high velocities, that is designed for aerodynamic efficiency, and that consists of pure lightweight construction, traditionally the target of the minimum lap time can only be achieved through extremely novel, sophisticated, and radical developments."

— Johannes Betz et al., *Autonomous Vehicles on the Edge: A Survey on Autonomous Vehicle Racing*, 2022 p. 1, [13]

Similarly, autonomous drone races act as a proving ground for advancements in AI and robotics. It fosters progress in perception, motion planning, and control in highly constrained environments. Success in this domain offers insights that can be transferred to broader applications for aerial robots and autonomy, which is why the relevance of this research extends beyond racing. In real-world applications such as infrastructure inspection, improved autonomy and speed can enhance efficiency. A recent survey on the use of UAVs

in bridge inspection found that most inspection drones rely on GPS navigation, with endurance, i.e., battery life, stated as the key limiting factor [16]. Additionally, the majority of drones employed by US Departments of Transportation are not fully autonomous and require skilled human pilots [16]. This illustrates that advancements made in the context of autonomous drone racing serve more conventional domains as well.

Beyond the scientific and practical motivation, there is a personal one, too. As a drone racing pilot myself, having competed in the German Drone Championship (DCS), I am highly motivated to the challenge of creating an autonomous racing system that can rival or even surpass human pilots' lap times. This project blends my academic background with my passion for the sport while pushing the limits of machine and human capability.

1.2 Contribution

This thesis presents a system for autonomous quadrotor navigation in a structured real-world environment using visual perception. We address the problem of autonomous drone racing by training a vision-based control policy in simulation and transferring it to a physical platform.

An artificial neural network is trained using Dataset Aggregation (DAgger) [7], with expert supervision provided by iterative discontinuity-bounded A* (iDbA*) [17], a novel motion planning algorithm that offers asymptotic optimality and excellent anytime performance.

To address the sim-to-real gap, we train a convolutional neural network to enable robust racing gate pose detection under varying visual conditions. The extracted image features are processed by a compact fully-connected controller network that maps our predicted gate pose vector to thrust and angular rate commands.

The final policy is deployed on the Bitcraze Crazyflie 2.1, a highly constrained UAV platform that weighs less than 40g. This makes the Crazyflie quadrotor comparable in size and form factor to "Tiny Whoop" racing drones, a separate discipline from conventional drone racing. To the best of our knowledge, the only autonomous racing platform of comparable size is "Trashcan", a quadrotor developed by researchers at TU Delft that weighs 72g [18].

The system is evaluated in a physical, real-world environment, where it demonstrates autonomous navigation through racing gates. Performance is benchmarked against a human pilot and an expert motion planner.

1.3 Thesis Overview

This thesis is structured into seven chapters, excluding appendices and references. A short description of each chapter is provided below:

Chapter 1: Introduction

This chapter outlines the motivation behind the derivation of the autonomous drone racing policy. The main contributions of the thesis are introduced, and a high-level overview of the thesis structure is provided.

Chapter 2: Background

This chapter presents foundational information required to understand the work. It covers quadrotor dynamics and control, motion planning algorithms such as iDbA* [17], and imitation learning techniques including dataset aggregation.

Chapter 3: Related Work

A survey of prior research conducted by other autonomous drone racing teams is provided in this chapter. It includes discussions on perception, planning, and control methods, as well as a review of the state of the art.

Chapter 4: Learning a Vision-to-Action Policy for Drone Racing

This chapter details the design and implementation of our vision-based autonomous drone racing policy. It describes the hardware and software setup and the approach taken regarding the autonomy pipeline.

Chapter 5: Evaluation and Experiments

The performance of our proposed system is evaluated in simulation and real-world hardware settings. Results are analyzed.

Chapter 6: Discussion

This chapter discusses the implications and limitations of the presented work and reflects on possible extensions and open challenges. Potential directions for future work are also outlined.

Chapter 7: Conclusion

The thesis concludes with a summary of the key contributions and findings.

2. Background

The field of autonomous drone racing combines a variety of scientific domains. For the derivation of an autonomous racing policy, quadrotor dynamics, motion planning, and control are specifically relevant. In this chapter, we provide an overview of the knowledge our research is based on, introducing core engineering concepts as well as providing a background on drone racing.

2.1 Drone Racing

Drone racing is a competitive and rapidly growing sport in which human pilots operate high-speed quadrotors through predefined obstacle courses that are marked by a sequence of gates. These courses demand precise maneuvering and high-speed decision-making.

The quadrotors are built for minimal weight, maximum thrust, and real-time responsiveness. The human operators send control inputs via a hand-held transmitter to the drone while visually perceiving the drone's state through a live video feed from an onboard camera.

Due to the difficulty of the sport, pilots practice for years in order to compete on a professional level. Hence, transitioning to autonomous drone racing poses significant challenges. An autonomous quadrotor must perceive its surroundings, plan collision-free and time-optimal trajectories, and control its motions. Speeds in autonomous drone races can exceed 20 m/s, with accelerations over 4g, which sets limitations for sensing, computation, and control capabilities [14]. The goal is to eventually outperform human pilots, reaching shorter lap times.

Autonomous drone racing serves as a benchmark for comparing hardware and software solutions for agile quadrotor flight. In addition to that, it serves as a proxy task for real-world high-speed robotic applications such as search and rescue, last-mile delivery, and infrastructure inspection. All of these require advances in state estimation, high-frequency trajectory planning, and robust control strategies [14].

Over the past years, a number of autonomous drone racing competitions have drawn attention to the field, including the Defense Advanced Research Projects Agency (DARPA) Fast Lightweight Autonomy program, the AlphaPilot Challenge by Lockheed Martin, and the International Conference on Intelligent Robots and Systems (IROS) Autonomous Drone Racing series. These events have served as testing grounds for comparing methods and performance under standardized conditions. In particular, the 2022 Swiss Drone Days event had autonomous racing policies defeat the best human FPV pilots in the world, while flying at speeds of over 22 m/s [14] [1].

As a complex interdisciplinary domain, autonomous drone racing progresses research in robotics, machine learning, real-time embedded systems, and autonomous navigation.

2.2 Quadrotor Dynamics

Modeling quadrotor dynamics provides a foundation on which control, motion planning, and thus autonomous drone racing can build. The goal is to derive a dynamics model that captures the quadrotors' physical characteristics and provides the dynamics computation for real-world control inputs such as roll, pitch, yaw, and thrust. In this section, we begin with a detailed representation of the full dynamics model, then demonstrate how the property of differential flatness allows us to simplify motion planning, and finally motivate the use of the double integrator as a reduced-order but effective model.

2.2.1 Full Dynamics Model

The quadrotor is a six-degree-of-freedom underactuated system. To model its full behavior, we must consider translational and rotational forces and moments on the drone's body, as well as their coupling through the robot's geometry and inertia [19].

We define the world frame W as an inertial frame and the body frame B as fixed to the quadrotor's center of mass. The orientation of the quadrotor is represented by a rotation matrix ${}^W R_B \in \text{SO}(3)$, which maps body frame vectors into world frame coordinates. Each of the four rotors $i \in \{1, 2, 3, 4\}$ generates a thrust force $F_i = k_F \omega_i^2$ and a moment

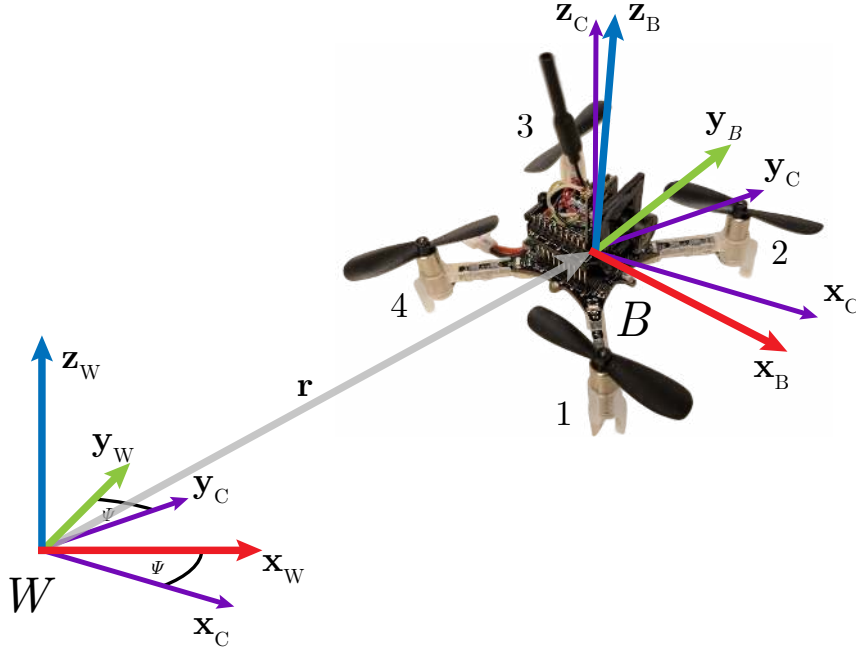


Figure 2.1: Visual representation of the flat-output frame C and reference frames. The inertial world frame W is shown at the origin, while the body frame B is fixed to the quadrotor's center of mass. Vector $\mathbf{r}$ indicates the position of the body-frame origin in world coordinates, and the rotors are numbered 1–4.

$M_i = k_M \omega_i^2$, where ω_i is the angular velocity of rotor i , and k_F, k_M are aerodynamic constants. Let L be the distance from each rotor to the center of mass. The total force and moments generated by the quadrotor can then be aggregated into a control input vector:

$$\mathbf{u} = \begin{bmatrix} u_1 \\ u_2 \\ u_3 \\ u_4 \end{bmatrix} = \begin{bmatrix} k_F & k_F & k_F & k_F \\ 0 & k_F L & 0 & -k_F L \\ -k_F L & 0 & k_F L & 0 \\ k_M & -k_M & k_M & -k_M \end{bmatrix} \begin{bmatrix} \omega_1^2 \\ \omega_2^2 \\ \omega_3^2 \\ \omega_4^2 \end{bmatrix} = \begin{bmatrix} \sum F_i \\ \tau_\phi \\ \tau_\theta \\ \tau_\psi \end{bmatrix}, \quad (2.1)$$

where u_1 is the net thrust in the body z_B direction, and u_2, u_3, u_4 represent moments about the x_B , y_B , and z_B axes, i.e., moments about roll, pitch, yaw.

The translational dynamics of the quadrotor's center of mass $\mathbf{r} \in \mathbb{R}^3$ in the world frame are governed by Newton's second law:

$$m\ddot{\mathbf{r}} = -mg\mathbf{e}_z + u_1\mathbf{z}_B, \quad (2.2)$$

where m is the mass of the quadrotor, g is gravitational acceleration, $\mathbf{e}_z$ is the unit vector in z -direction, and $\mathbf{z}_B = {}^W R_B \mathbf{e}_z$ is the body z -axis in world coordinates, representing the thrust direction.

The rotational dynamics are governed by the Euler equations:

$$\dot{\boldsymbol{\omega}} = I^{-1} \left(-\boldsymbol{\omega} \times I\boldsymbol{\omega} + \begin{bmatrix} u_2 \\ u_3 \\ u_4 \end{bmatrix} \right), \quad (2.3)$$

where $\boldsymbol{\omega} \in \mathbb{R}^3$ is the angular velocity of the body frame with respect to the world frame, expressed in the body frame, and $I \in \mathbb{R}^{3 \times 3}$ is the moment of inertia matrix.

The full state vector of the quadrotor can be expressed as:

$$\mathbf{x} = [\mathbf{r}, \mathbf{q}_\phi, \dot{\mathbf{r}}, \boldsymbol{\omega}]^\top, \quad (2.4)$$

with $\mathbf{q}_\phi = [q_w, q_x, q_y, q_z]^\top$ containing the rotational information in a quaternion, where q_w is the scalar part and q_x, q_y, q_z represent the vector part of the quaternion.

This model provides the foundation for nonlinear control of quadrotors even during aggressive maneuvers, where roll and pitch may deviate significantly from zero and linearizations around hover are insufficient [19].

However, planning trajectories directly in this high-dimensional, nonlinear space is computationally complex.

2.2.2 Differential Flatness

The concept of differential flatness can allow for a simplified parametrization of the full state of the robot using a lower-dimensional set of outputs and their derivatives. A system governed by dynamics $\dot{\mathbf{x}} = \mathbf{f}(\mathbf{x}, \mathbf{u})$ is said to be differentially flat if there exists a set of outputs $\mathbf{z}(t)$, called flat outputs so that the state $\mathbf{x}(t)$ and the input $\mathbf{u}(t)$ can be written in terms of $\mathbf{z}(t)$ and a finite number of its derivatives [20]:

$$\mathbf{x}(t) = g_q(\mathbf{z}, \dot{\mathbf{z}}, \ddot{\mathbf{z}}, \dots), \quad (2.5)$$

$$\mathbf{u}(t) = g_u(\mathbf{z}, \dot{\mathbf{z}}, \ddot{\mathbf{z}}, \dots). \quad (2.6)$$

In our case, the flat outputs are chosen as:

$$\boldsymbol{\sigma}(t) = [x(t), y(t), z(t), \psi(t)]^\top. \quad (2.7)$$

This choice captures the translational motion and yaw angle while encoding all of the state information through derivatives. Using this property, we can generate smooth trajectories in the flat output space, from which the internal states position $\mathbf{r}$, velocity $\dot{\mathbf{r}}$, orientation quaternion $\mathbf{q}_\phi$, as well as the control input collective thrust u_1 can be recovered [21]. We will illustrate this further in the next section.

2.2.3 Double Integrator Model

The following demonstrates that the double integrator model can be utilized as an effective, sparse approximation of the quadrotors' full dynamics model described in section 2.2.1.

The translational acceleration $\ddot{\boldsymbol{\sigma}}$ determines the net thrust direction, as seen from the equation of motion.

Rearranging (2.2) gives:

$$\mathbf{t} = m\ddot{\boldsymbol{\sigma}} + mg\mathbf{e}_z, \quad \mathbf{z}_B = \frac{\mathbf{t}}{\|\mathbf{t}\|}, \quad (2.8)$$

where $\mathbf{z}_B$ is the direction of thrust and $\mathbf{t} = [\ddot{\sigma}_1, \ddot{\sigma}_2, \ddot{\sigma}_3 + g]^T$. To resolve the full orientation, the desired yaw ψ defines the direction $\mathbf{x}_C$ in the world frame:

$$\mathbf{x}_C = \begin{bmatrix} \cos \psi \\ \sin \psi \\ 0 \end{bmatrix}, \quad (2.9)$$

From this, the remaining body-frame axes are constructed as:

$$\mathbf{y}_B = \frac{\mathbf{z}_B \times \mathbf{x}_C}{\|\mathbf{z}_B \times \mathbf{x}_C\|}, \quad \mathbf{x}_B = \mathbf{y}_B \times \mathbf{z}_B. \quad (2.10)$$

The full orientation is then:

$${}^wR_B = [\mathbf{x}_B \quad \mathbf{y}_B \quad \mathbf{z}_B]. \quad (2.11)$$

Finally, the net thrust u_1 is computed as:

$$u_1 = m\|\ddot{\boldsymbol{\sigma}} + g\mathbf{e}_z\| = m\mathbf{z}_B^\top(\ddot{\boldsymbol{\sigma}} + g\mathbf{e}_z). \quad (2.12)$$

Thus, differential flatness provides a direct mapping from trajectory derivatives to control input u_1 .

Since the flat outputs $\boldsymbol{\sigma}(t)$ evolve in $\mathbb{R}^3 \times \text{SO}(2)$, and the control inputs map to their derivatives, the translational dynamics of a quadrotor can be abstracted as a second-order system:

$$\dot{\mathbf{r}} = \mathbf{v}, \quad (2.13)$$

$$\ddot{\mathbf{r}} = \mathbf{u}, \quad (2.14)$$

where $\mathbf{r} \in \mathbb{R}^3$ is the position, $\dot{\mathbf{r}} \in \mathbb{R}^3$ is the velocity, and $\mathbf{u} \in \mathbb{R}^3$ represents the control input, acceleration.

This double integrator model provides a sparse computation of the robot's differentially flat state. It acts as a bridge between the complex dynamics model and computationally efficient planning algorithms.

2.3 Quadrotor Control

Quadrotors are inherently unstable, nonlinear, coupled, and under-actuated systems. These properties make them well-suited for studying and validating control theories. Control strategies for quadrotors can be grouped into two main categories: geometric and optimization-based controllers [22] [23].

Geometric controllers exploit the quadrotors' dynamics model to derive both classical linear control strategies, including PID, LQR, and H_∞ , and nonlinear methods like feedback linearization and sliding-mode control. These approaches offer simplicity and low computational load, but may struggle under aggressive maneuvers. In contrast, optimization-based controllers, such as model predictive control (MPC), formulate flight control as a real-time, finite-horizon optimal control problem. At each timestep, MPC predicts the system's future states over a specified horizon and computes optimal control inputs by minimizing a cost function that is subject to the quadrotors' dynamics. However, MPCs' improved control characteristics induce higher computational effort.

In this section, we present a nonlinear control strategy for tracking trajectories generated in the differentially flat output space we proposed [19], as well as highlight other geometric control strategies, such as PID-control. Additionally, we provide background optimization-based control strategies such as MPC and reinforcement learning control.

2.3.1 Nonlinear Controller Derivation

As controller setpoints, we define the desired translational path and yaw angle by specifying a trajectory as $\boldsymbol{\sigma}_T(t) = [x_T(t), y_T(t), z_T(t), \psi_T(t)]^\top$. Using the inverse dynamics of the quadrotor (2.26), we can then compute the motor thrusts required to track the trajectory.

Let $\mathbf{r}_T$ and $\dot{\mathbf{r}}_T$ be the desired position and velocity. The position and velocity errors are then defined as:

$$\mathbf{e}_p = \mathbf{r} - \mathbf{r}_T, \quad \mathbf{e}_v = \dot{\mathbf{r}} - \dot{\mathbf{r}}_T. \quad (2.15)$$

Next, the desired force vector is computed as:

$$\mathbf{F}_{\text{des}} = -K_p \mathbf{e}_p - K_v \mathbf{e}_v + m g \mathbf{e}_z + m \ddot{\mathbf{r}}_T, \quad (2.16)$$

where $K_p, K_v \in \mathbb{R}^{3 \times 3}$ are positive definite gain matrices. The direction of the desired body z -axis is defined as:

$$\mathbf{z}_{B,\text{des}} = \frac{\mathbf{F}_{\text{des}}}{\|\mathbf{F}_{\text{des}}\|}, \quad (2.17)$$

and the net thrust input becomes:

$$u_1 = \mathbf{F}_{\text{des}} \cdot \mathbf{z}_B. \quad (2.18)$$

To define the desired orientation, we introduce the yaw-aligned vector in the world frame:

$$\mathbf{x}_{C,\text{des}} = \begin{bmatrix} \cos \psi_T \\ \sin \psi_T \\ 0 \end{bmatrix}. \quad (2.19)$$

Using this, the remaining desired body axes are defined similarly to (2.10) as:

$$\mathbf{y}_{B,\text{des}} = \frac{\mathbf{z}_{B,\text{des}} \times \mathbf{x}_{C,\text{des}}}{\|\mathbf{z}_{B,\text{des}} \times \mathbf{x}_{C,\text{des}}\|}, \quad (2.20)$$

$$\mathbf{x}_{B,\text{des}} = \mathbf{y}_{B,\text{des}} \times \mathbf{z}_{B,\text{des}}, \quad (2.21)$$

and the desired rotation matrix is given by:

$${}^W R_{B,\text{des}} = \begin{bmatrix} \mathbf{x}_{B,\text{des}} & \mathbf{y}_{B,\text{des}} & \mathbf{z}_{B,\text{des}} \end{bmatrix}. \quad (2.22)$$

The orientation error is then computed using the vee map $(\cdot)^\vee$:

$$\mathbf{e}_R = \frac{1}{2} (R_{\text{des}}^\top {}^W R_B - {}^W R_B^\top R_{\text{des}})^\vee, \quad (2.23)$$

This formulation avoids the singularities and discontinuities associated with Euler angles [24].

The angular velocity error is computed in the body frame as:

$$\mathbf{e}_\omega = {}^B[\boldsymbol{\omega}_{BW}] - {}^B[\boldsymbol{\omega}_{BW,T}] \quad (2.24)$$

The control torques are computed using a proportional-derivative (PD) control law on these errors:

$$\begin{bmatrix} u_2 \\ u_3 \\ u_4 \end{bmatrix} = -K_R \mathbf{e}_R - K_\omega \mathbf{e}_\omega, \quad (2.25)$$

where K_R and K_ω are diagonal gain matrices. Finally, the required squared rotor speeds are calculated by inverting the matrix (2.1) mapping rotor thrusts and moments to control inputs:

$$\begin{bmatrix} \omega_1^2 \\ \omega_2^2 \\ \omega_3^2 \\ \omega_4^2 \end{bmatrix} = \begin{bmatrix} k_F & k_F & k_F & k_F \\ 0 & k_F L & 0 & -k_F L \\ -k_F L & 0 & k_F L & 0 \\ k_M & -k_M & k_M & -k_M \end{bmatrix}^{-1} \begin{bmatrix} u_1 \\ u_2 \\ u_3 \\ u_4 \end{bmatrix}. \quad (2.26)$$

This final mapping assumes motor thrusts and torques can be achieved instantaneously.

2.3.2 PID Control

Having abstracted the quadrotor's translational and yaw dynamics as three independent double integrators

$$\dot{\mathbf{r}} = \mathbf{v}, \quad \ddot{\mathbf{r}} = \mathbf{u},$$

where $\mathbf{r} = [x, y, z]^\top$, and $\mathbf{u} = \ddot{\mathbf{r}}$ represents the commanded acceleration, we can apply a PID law to each channel of this model.

Let the reference trajectory be given by the flat outputs $\boldsymbol{\sigma}_{\text{ref}}(t) = [x_{\text{ref}}(t), y_{\text{ref}}(t), z_{\text{ref}}(t), \psi_{\text{ref}}(t)]^\top$. We can then denote the translational error on axis $j \in \{x, y, z\}$ as

$$e_j(t) = r_{j,\text{ref}}(t) - r_j(t),$$

and the corresponding velocity error as

$$\dot{e}_j(t) = \dot{r}_{j,\text{ref}}(t) - \dot{r}_j(t).$$

A PID controller for each translational axis then computes the commanded acceleration

$$u_j(t) = K_{P,j} e_j(t) + K_{I,j} \int_0^t e_j(\tau) d\tau + K_{D,j} \dot{e}_j(t). \quad (2.27)$$

To close the loop, this $\mathbf{u}$ is mapped through the flat-output mapping back to thrust and attitude commands:

$$u_1 = m \|\mathbf{u} + g \mathbf{e}_z\|,$$

as described in Section 2.2.2.

For yaw, a PI controller suffices:

$$\tau_\psi(t) = K_{P,\psi} (\psi_{\text{ref}}(t) - \psi(t)) + K_{I,\psi} \int_0^t (\psi_{\text{ref}}(\tau) - \psi(\tau)) d\tau.$$

Quadrotor flight stacks often employ a cascaded PID architecture, nesting loops for position, velocity, attitude, and body-rate control [25]. The outer loops generate intermediate setpoints, which feed the inner loops that stabilize angular rates. This separation of time scales improves disturbance rejection and enables independent tuning of each loop.

Classical tuning methods (e.g. Ziegler–Nichols) have been widely used in early quadrotor control, yielding stable hover and satisfactory transient response [26]. More systematic approaches, such as genetic algorithm optimization of PID gains, have demonstrated further reductions in settling time and integral error, particularly under external disturbances [27].

2.3.3 Model Predictive Control

Model predictive control (MPC) is a control strategy that optimizes a sequence of future control inputs to minimize a performance indicator while enforcing system dynamics and input and state constraints. The discrete-time nonlinear dynamics can be written as

$$\mathbf{x}_{k+1} = f(\mathbf{x}_k, \mathbf{u}_k), \quad (2.28)$$

where $x_k = [\mathbf{r}_k, \mathbf{q}_{\phi,k}, \dot{\mathbf{r}}_k, \boldsymbol{\omega}_k]^\top$ contains position, attitude, velocity and angular rates, and $\mathbf{u}_k$ the rotor thrusts and torques. A linear MPC implementation linearizes these dynamics, yielding

$$\mathbf{x}_{k+1|k} = A \mathbf{x}_{k|k} + B \mathbf{u}_{k|k}, \quad (2.29)$$

$$\min_{\{\mathbf{u}_{k+i|k}\}_{i=0}^{N-1}} J = \sum_{i=0}^{N-1} \|\mathbf{x}_{k+i|k} - \mathbf{x}_{\text{ref}, k+i}\|_Q^2 + \|\mathbf{u}_{k+i|k}\|_R^2 \quad (2.30)$$

subject to

$$\mathbf{u}_{\min} \leq \mathbf{u}_{k+i|k} \leq \mathbf{u}_{\max}. \quad (2.31)$$

This formulation expresses a finite-horizon quadratic program in discrete time, where $\mathbf{x}_{k+i|k}$ and $\mathbf{u}_{k+i|k}$ are the predicted state and control input at step $k+i$ based on information at k . A and B are the linearized system matrices, $\mathbf{x}_{\text{ref}, k+i}$ is the desired trajectory, Q and R weight state tracking versus control effort, and $\mathbf{u}_{\min}, \mathbf{u}_{\max}$ enforce actuator limits [28], [29].

Early onboard MPC implementations demonstrated that quadrotor position and trajectory tracking can be achieved under actuator constraints [28].

2.3.4 Reinforcement Learning Control

Reinforcement learning (RL) formulates quadrotor control as a Markov decision process $(\mathcal{S}, \mathcal{A}, P, r, \gamma)$, where $\mathcal{S}$ is the set of states, $\mathcal{A}$ is the set of actions, $P(\mathbf{s}_{t+1} | \mathbf{s}_t, \mathbf{a}_t)$ is the transition probability, $r(\mathbf{s}_t, \mathbf{a}_t) \in \mathbb{R}$ is the scalar reward received for taking action $\mathbf{a}_t$ in state $\mathbf{s}_t$, and $\gamma \in [0, 1)$ is the discount factor that balances the importance of immediate versus future rewards.

A stochastic policy $\pi_\theta(\mathbf{a} | \mathbf{s})$, parameterized by θ , defines a distribution over actions given the current state. The objective is to maximize the expected discounted return

$$J(\theta) = \mathbb{E}_{\pi_\theta} \left[\sum_{t=0}^T \gamma^t r(\mathbf{s}_t, \mathbf{a}_t) \right],$$

where T is the episode horizon. In practice, the policy parameters are adjusted via gradient-based RL algorithms such as policy gradients or actor-critic algorithms, which maximize the expected return.

In the quadrotor setting, the state is typically defined as

$$\mathbf{s}_t = [\mathbf{e}_p(t), \mathbf{e}_v(t), \mathbf{e}_R(t), \mathbf{e}_\omega(t)],$$

where $\mathbf{e}_p = \mathbf{p} - \mathbf{p}_d$ is the position error, $\mathbf{e}_v = \mathbf{v} - \mathbf{v}_d$ the velocity error, $\mathbf{e}_R = \text{vee}(R_d^\top R - I)$ the attitude error expressed via the vee-map, and $\mathbf{e}_\omega = \boldsymbol{\omega} - \boldsymbol{\omega}_d$ the angular-rate error [30]. The action is defined as

$$\mathbf{a}_t = [f_T, \mathbf{M}],$$

with f_T denoting total thrust and $\mathbf{M} \in \mathbb{R}^3$ the body-fixed torque vector. A common choice of reward is the negative weighted sum of errors,

$$r(\mathbf{s}_t, \mathbf{a}_t) = -(\lambda_p \|\mathbf{e}_p\| + \lambda_v \|\mathbf{e}_v\| + \lambda_R \|\mathbf{e}_R\| + \lambda_\omega \|\mathbf{e}_\omega\|),$$

where each weight $\lambda_p, \lambda_v, \lambda_R, \lambda_\omega > 0$ trades off the importance of position accuracy, velocity tracking, attitude alignment, and rate damping, and γ determines how far into the future the controller considers the accumulated reward.

An example of this paradigm is presented in [30], where an RL policy was trained in simulation to minimize the composite error reward and was deployed on hardware. The researchers found that the RL controller achieved faster stabilization and improved disturbance rejection compared to geometric controllers.

2.4 Motion Planning

Motion planning for robotic systems with complex dynamics is a fundamental challenge. This applies in particular to autonomous drone racing, where trajectories must satisfy dynamic and environmental constraints.

Several groups of approaches have been proposed for kinodynamic motion planning. Among these, sampling-based planners [31] [32] [33] are widely used due to their ability to explore high-dimensional state spaces and their theoretical guarantees. These methods typically construct trees by randomly propagating states. Their convergence to optimal solutions can be slow in practice, and the resulting paths may require additional processing to satisfy dynamic constraints.

Search-based methods [34] [35] use precomputed motion primitives to construct discrete graphs over the state space. While these approaches offer theoretical guarantees, they often require manual design of the motion primitives set. Additionally, search-based methods do not scale well with increasing system complexity or dimensionality.

Optimization-based methods [36] [37] [38] formulate trajectory generation as an optimization problem. These methods can produce smooth, dynamically valid paths. The resulting trajectories are only locally optimal and typically require an initial guess that is close to feasibility.

To mitigate the limitations of the individual classes, hybrid approaches have been introduced that combine sampling, search, and optimization techniques [17]. These methods leverage the exploration capabilities of sampling, the theoretical guarantees of discrete planners, and the smooth, dynamically valid paths offered by optimization.

2.4.1 iDbA* Algorithm

iDb-A* is a hybrid motion planning algorithm that combines sampling, search, and optimization-based strategies to solve kinodynamic planning problems. iDb-A*'s strength lies in quickly finding asymptotically optimal solution trajectories while adhering to the robots' dynamics.

iDbA* is an extension of the discontinuity-bounded A* (Db-A*) algorithm [39], which is a modified version of the classic A* search algorithm tailored for kinodynamic planning using motion primitives. These primitives act as edges of an implicit graph in continuous space and adhere to the robot's dynamics. Db-A* does not require alignment between the end state of one primitive and the start state of the next. Instead, it allows connections with bounded discontinuities in state space. The size of the discontinuities can be controlled by a parameter δ . Hence, this threshold defines the maximum deviation allowed when connecting motion primitives. The search is structured to avoid redundant expansion by treating states within a $(1 - \alpha)\delta$ radius as equivalent. A trajectory returned by DbA* is infeasible with respect to the system dynamics due to the discontinuity bounds, but it is a useful approximation that can be refined.

Algorithm 1: iDb-A* – Iterative Discontinuity Bounded A*

Input: Start state x_s , Goal state x_g , Step size $step$, Free space $\mathcal{X}_{\text{free}}$, Control set $\mathcal{U}$, Motion primitives $\mathcal{M}_L$

Output: Trajectory X , Control sequence U

```

1 Initialize empty primitive set  $\mathcal{M}_0 \leftarrow \emptyset$ 
2 Initialize cost bound  $c_{\max} \leftarrow \infty$ 
3 for  $i = 1, 2, \dots$  do
4   Augment Primitives  $\mathcal{M}_i \leftarrow \mathcal{M}_{i-1} \cup \text{AddPrimitives}(\mathcal{M}_L, i)$ 
5   Update discontinuity threshold  $\delta_i \leftarrow \text{DecreaseDelta}(i)$ 
6   Run bounded A* search  $(X_d, U_d) \leftarrow \text{DbA}^*(x_s, x_g, \mathcal{X}_{\text{free}}, \mathcal{M}_i, \delta_i, c_{\max})$ 
7   if  $(X_d, U_d \text{ successfully computed})$  then
8     Optimize Traj.  $(X, U) \leftarrow \text{Optimization}(X_d, U_d, x_s, x_g, step, \mathcal{X}_{\text{free}}, \mathcal{U})$ 
9     if  $(X, U \text{ successfully computed})$  then
10      Output a new feasible solution  $\text{Report}(X, U)$ 
11      Tighten cost bound  $c_{\max} \leftarrow \min(c_{\max}, J(X, U))$ 
12      Extract primitives from solution  $\mathcal{M}_i \leftarrow \mathcal{M}_i \cup \text{ExtractPrimitives}(X, U)$ 

```

Building on this foundation, iDb-A* wraps Db-A* in an outer iterative loop that progressively improves solution quality. As implemented in algorithm 1, for each iteration, the number of available motion primitives is increased and the discontinuity bound δ is decreased, which makes the discontinuous states move closer until they converge towards continuity.

Initially, the discrete search performed by Db-A* will output a candidate trajectory that is used to start a trajectory optimization step. This optimization procedure locally adjusts the discontinuous path into a dynamically feasible and smooth trajectory by minimizing a cost function. In the next step, new motion primitives are extracted from

the optimized trajectory and added to the set of primitives, providing the next iteration with goal-oriented state-lattices.

This iterative process continues, progressively tightening the discontinuity tolerance and enhancing the motion primitive set, ultimately converging to a locally optimal solution. iDb-A* is an efficient anytime planner that balances exploration and exploitation. The Algorithm explores the state space initially to find non-optimal goal paths quickly, and has later iterations improve solution quality. Additionally, the approach does not require a fixed discretization of the state space and avoids reliance on handcrafted motion primitives.

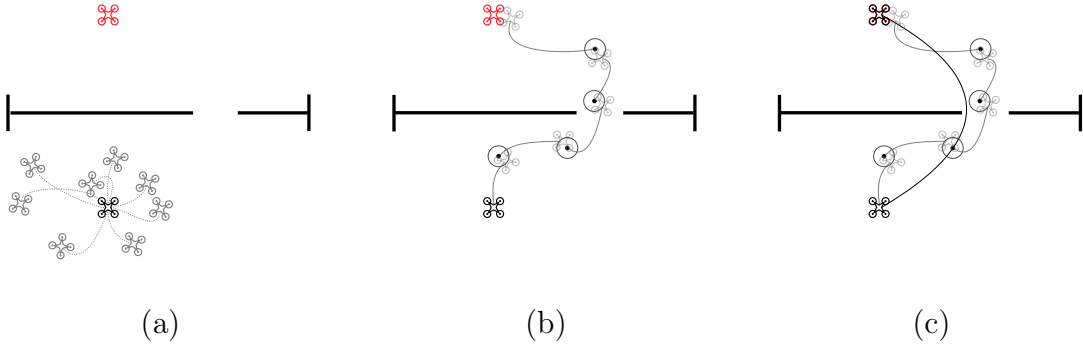


Figure 2.2: Visual representation of a 2D quadrotor model’s behavior navigating through a gate using the iDbA* algorithm during its first iteration. Start and goal states of the planning are indicated with black (start) and red (goal) quadrotor symbols. (a) Initial expansion of motion primitives from the start configuration. (b) Solution trajectory found by DbA*. The path illustrates the discontinuity-bounded motion primitives stitched together. (c) Smoothed and dynamically feasible trajectory obtained by post-processing the initial solution using optimization.

The property to find locally optimal goal paths in a comparably short amount of time [17] makes iDb-A* a compelling choice for generating high-quality expert demonstrations in imitation learning pipelines for drone racing policies.

2.4.2 Rapidly-Exploring Random Trees

The sampling-based motion planner Rapidly-Exploring Random Trees (RRT) is integrated into a variety of robotic applications, such as autonomous cars, surveillance operations, agricultural robots, planetary and space exploration missions [40]. Due to RRTs’ ability to handle high-dimensional configuration spaces and kinodynamic constraints, RRT variants are implemented in quadrotor motion planning frameworks [41]. As shown in algorithm 2, RRT incrementally constructs a tree by randomly sampling a configuration $\mathbf{x}_{\text{rand}}$, selecting the nearest existing node $\mathbf{x}_{\text{near}}$, and extending toward $\mathbf{x}_{\text{rand}}$ under collision and feasibility checks [42]. Such sampling-based frameworks often serve as a front-end to trajectory smoothing and optimization methods [17].

For instance, in [43] researchers employed a bi-directional RRT to compute collision-free paths for the quadrotor’s center of mass in tightly constrained environments and then applied a local kinodynamic planner to convert these geometric paths into smooth, dynamically feasible trajectories.

Algorithm 2: RRT: Rapidly-Exploring Random Tree Generation

Input: Initial state $\mathbf{x}_{\text{init}}$, number of iterations K , time step Δt
Output: Tree $\mathcal{T}$ spanning reachable states from $\mathbf{x}_{\text{init}}$

- 1 Initialize tree $\mathcal{T}$ with root node at $\mathbf{x}_{\text{init}}$
- 2 **for** $k = 1$ **to** K **do**
- 3 Sample a random state $\mathbf{x}_{\text{rand}}$
- 4 Find nearest node $\mathbf{x}_{\text{near}}$ to $\mathbf{x}_{\text{rand}}$ in $\mathcal{T}$
- 5 Compute control input $\mathbf{u}$ that drives $\mathbf{x}_{\text{near}}$ toward $\mathbf{x}_{\text{rand}}$
- 6 Simulate system dynamics from $\mathbf{x}_{\text{near}}$ using $\mathbf{u}$ and step size Δt , yielding $\mathbf{x}_{\text{new}}$
- 7 **if** $\mathbf{x}_{\text{new}}$ is collision-free and feasible **then**
- 8 Add $\mathbf{x}_{\text{new}}$ to $\mathcal{T}$ as a vertex
- 9 Add edge from $\mathbf{x}_{\text{near}}$ to $\mathbf{x}_{\text{new}}$ labeled with input $\mathbf{u}$
- 10 **return** $\mathcal{T}$

2.5 Imitation Learning

Imitation learning is a paradigm within machine learning that focuses on teaching agents to perform tasks by observing expert demonstrations.

The roots of imitation learning originate from observations of biological systems. Humans and animals usually learn skills by mimicking others, thereby using demonstrations as a way to narrow the search space for effective behaviors [44]. From an engineering perspective, imitation learning addresses the need for exploration and well-defined reward signals, which are crucial for paradigms like reinforcement learning. Imitation learning bypasses these needs and can learn desirable behaviors from a comparably small number of examples [44].

However, imitation learning shows limitations regarding the mismatch between the distribution of states observed during demonstrations and those encountered when the learned policy is deployed. This issue is known as distributional shift, which can lead to compounding errors during execution. Imitation learning algorithms that can adaptively gather training data from the student’s experience address this problem. This principle is implemented in DAgger, which outperforms other popular imitation learning algorithms [7].

2.5.1 Dataset Aggregation

Traditionally, imitation learning is formulated as a supervised learning problem. A policy $\hat{\pi}$ is trained to minimize the expected surrogate loss over the distribution of states d_{π^*} induced by the expert policy π^* :

$$\hat{\pi} = \arg \min_{\pi \in \Pi} \mathbb{E}_{\mathbf{s} \sim d_{\pi^*}} [\ell(\mathbf{s}, \pi)]. \quad (2.32)$$

Here, $\ell(\mathbf{s}, \pi)$ denotes the loss incurred by policy π when acting in state $\mathbf{s}$, and Π is the class of policies under consideration.

This supervised learning formulation does not account for distributional shift. The policy is optimized under the expert’s state distribution d_{π^*} , but at deployment, the learned policy encounters its own distribution of states $d_{\hat{\pi}}$. Since small prediction errors can cause

the agent to visit states not seen during training, the errors can compound over time, which results in degraded policy performance. In particular, it can be shown that the cost of executing $\hat{\pi}$ grows quadratically with T -steps and the per-step error rate ϵ , which leads to a performance bound of $O(T^2\epsilon)$ [7].

The pseudo-code in Algorithm 3 demonstrates DAgger, a policy training algorithm that addresses these issues. The iterative imitation learning paradigm reduces distributional shift by collecting training data under the learner’s evolving policy.

Algorithm 3: DAgger Algorithm

Input: Expert policy π^* , policy class Π , iterations N , horizon T

Output: Learned policy $\hat{\pi}$ with best validation performance

```

1  $\mathcal{D} \leftarrow \emptyset$ 
2  $\hat{\pi}_1 \leftarrow$  any policy in  $\Pi$ 
3 for  $i = 1$  to  $N$  do
4    $\pi_i \leftarrow \beta_i \pi^* + (1 - \beta_i) \hat{\pi}_i$ 
5   Sample  $T$ -step trajectories using  $\pi_i$ 
6    $\mathcal{D}_i \leftarrow \{(\mathbf{s}, \pi^*(\mathbf{s}))\}$  of states visited by  $\pi_i$ 
7    $\mathcal{D} \leftarrow \mathcal{D} \cup \mathcal{D}_i$ 
8   Train classifier  $\hat{\pi}_{i+1}$  on  $\mathcal{D}$ 
9 return best  $\hat{\pi}_i$  on validation

```

The key idea behind DAgger is to iteratively gather data not only from the expert’s distribution but also from the distribution induced by the student’s current policy. By allowing the policy to interact with the environment during training and continually aggregating expert-labeled states encountered during execution, DAgger can handle the distributional shift.

The theoretical analysis of DAgger shows that it achieves a much stronger performance guarantee than the standard supervised approach. In particular, if the per-step classification error remains bounded by ϵ , the expected total cost of the learned policy grows only linearly with T -steps, leading to a performance bound of $O(T\epsilon)$ [7].

2.5.2 Generative Adversarial Imitation Learning

Generative adversarial imitation learning (GAIL) frames imitation as an adversarial game between a generative policy π_θ and a discriminator D_ϕ . At each iteration, the discriminator is trained to distinguish expert state–action pairs $(\mathbf{s}, \mathbf{a}) \sim d_{\pi^*}$ from those generated by the learner $(\mathbf{s}, \mathbf{a}) \sim \pi_\theta$, while the policy is updated to minimize this discrimination [45]:

$$\min_{\theta} \max_{\phi} \mathbb{E}_{(\mathbf{s}, \mathbf{a}) \sim d_{\pi^*}} [\log D_\phi(\mathbf{s}, \mathbf{a})] + \mathbb{E}_{(\mathbf{s}, \mathbf{a}) \sim \pi_\theta} [\log(1 - D_\phi(\mathbf{s}, \mathbf{a}))].$$

The discriminator $D_\phi(\mathbf{s}, \mathbf{a}) \in [0, 1]$ is a binary classifier parameterized by ϕ that estimates the probability that the pair $(\mathbf{s}, \mathbf{a})$ originates from the expert dataset d_{π^*} . The inner maximization over ϕ trains D_ϕ to assign high scores to expert samples and low scores to policy samples, while the outer minimization over θ updates π_θ to convince the discriminator it’s the expert D_ϕ , effectively matching the student policies’ behavior to that of the expert.

GAIL has been applied to train quadrotors for navigation within a simulated environment [46]. By imitating expert flight trajectories, the learned policy acquires the ability to interpret and adapt to complex, dynamic scenes. The approach enables a student policy to generalize across a variety of virtual scenarios, maintaining robust, target-oriented obstacle avoidance.

3. Related Work

The performance of autonomous drone racing policies increases continuously due to research that progresses hard and software. Recently, researchers hit a milestone where autonomous systems have completed racing laps in times close to those of professional human pilots [1]. In this chapter, we identify the main contributors to the field as well as the principles applied in state-of-the-art publications for the successful implementation of autonomous drone racing.

3.1 Autonomous Drone Racing

In conventional robotics architectures, the autonomous navigation task is decomposed into three fundamental components: Perception, Planning, and Control [14]. This section reviews recent advancements in each of these fields, with a focus on agile flight and autonomous drone racing. All methods discussed here require a prior formulation of quadrotor dynamics, which we provided in section 2.2.



Figure 3.1: A classic architecture for autonomous drone racing systems consisting of sensors and the drone as hardware components and perception, planning, and control as software stack.

3.1.1 State of the Art

We now assess the state of the art for autonomous drone racing policies. To that end, we evaluate UAV laboratories' publications. The research teams to be investigated are chosen based on their performance in recent races.

Alphapilot

The 2019 AlphaPilot Challenge provides a recent comparison for autonomous drone racing frameworks. The competition is particularly relevant because the majority of qualified teams have published their results. In the challenge, each competitor was provided with identical hardware to complete a race track. Among over 400 participating and nine qualified teams, the University of Zurich, UZH, and TU Delft MavLab emerged as the top performers, with TU Delft winning the challenge and Zurich placing second. Both teams published detailed reports of their successful approaches [47] [11], which are the main focus of this chapter.

In addition to their AlphaPilot results, both teams have continued to demonstrate outstanding autonomous racing performance. The UZH team hit a milestone at the 2022 Swiss Drone Days, at which their autonomous policy, Swift, outperformed human pilots in terms of lap times [1].

TU Delft MavLab further showed impressive results by winning the 2025 A2RL Drone Racing Championship in Abu Dhabi, which highlights their continued leadership in the field. As of July 2025, they have not published which techniques they used to succeed.

Honorable mentions include team Icarus of Georgia Institute of Technology, team USRG of Korea Advanced Institute of Science and Technology, and team XQuad of Universidade Federal de Minas Gerais. All teams made it to the top nine participants of the Alphapilot challenge and provided their methods in recent publications [48] [49] [50].

While we focus on the teams' recent findings, the following sections will also cover traditional methods for autonomous navigation of aerial robots.

Team Name	Input	Perception	Planning	Control	Output
MavLab ¹⁾	RGB Cam, IMU	GateNet (CNN)	Gate-aligned waypoint strategy	Feedforward and PID vel. ctrl	Angular rates, thrust
UZH ²⁾	RGB Cam, IMU	CNN corner det., VIO	Sampling-based planner	PD pos. + P att. ctrl	Angular rates, thrust
Icarus ³⁾	RGB Cam	YOLOv3	PixelMPC (MPC)	MPPI	Angular rates, thrust
USRG ⁴⁾	Depth Camera, IMU, Opt. Flow Sensor	ADNet (CNN)	LOS-based guidance	Roll- and yaw-rate ctrl	Angular rates, thrust
XQuad ⁵⁾	Depth Camera, IMU	YOLOv4, VIO	Offline polynomial traj.	Field-based nonlin. ctrl	Angular rates, thrust

Table 3.1: Comparison of different teams' architectures across the autonomy pipeline. The selection was based on their recent successes in the 2019 Alphapilot and 2025 AR2L drone racing competitions. The teams associated institutions are: 1) Team MavLab [11], Delft University of Technology, 2) Team UZH [47], University of Zurich, 3) Team Icarus [48], Georgia Institute of Technology, 4) Team USRG [49], Korea Advanced Institute of Science and Technology, 5) Team XQuad [50], Universidade Federal de Minas Gerais

3.1.2 Perception

Accurate perception and state estimation are critical in the context of drone racing since errors in perception negatively affect planning and control and thus lead to degraded performance. The predominant approach for onboard state estimation is Visual-Inertial Odometry (VIO) [51]. Inertial measurement units (IMU) provide translational acceleration, angular velocity, and magnetic field strength, usually in three degrees of freedom. Sensors can be fused to estimate position and orientation, but suffer from drift over time [52]. Camera measurements provide environmental information but are sensitive to challenging visual conditions, like illumination or motion blur. Together, the sensors provide complementary strengths that make VIO a reliable non-learning-based perception method for aerial robots [51].

Direct methods work directly on raw pixel intensities and extract image patches to estimate the cameras' trajectory by tracking the motion of these patches through consecutive images [53]. These methods are especially effective in low-texture environments where feature extraction is unreliable, which was shown by team Icarus, who deployed this perception strategy [48]. *Feature-based* methods identify and track visual key points across frames, offering greater robustness to variations in lighting and brightness conditions [54].

A typical VIO pipeline consists of a front end and a back end. The front end estimates the motion of the camera using visual input. The VIO back end is responsible for sensor fusion of IMU and camera data. From the IMU data, a motion prior is computed, and the camera-derived constraints from feature tracking correct the drift and scale ambiguity. The fusion is done via filtering, typically using an Extended Kalman Filter (EKF) or the Multi-State Constraint Kalman Filter (MSCKF) algorithm [55]. Because of its balance between computational cost and accuracy, MSCKFs have been adopted in high-speed drone applications [56]. Recent advances in state estimation have introduced other sensors

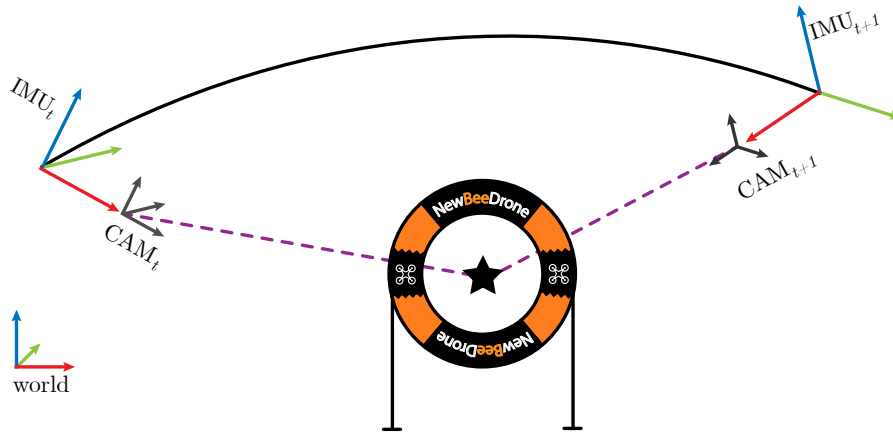


Figure 3.2: Visual representation of the visual-inertial odometry principle. The IMU frame (which aligns with the drone's body frame) and camera frame move from an initial pose (left) at $t = n$ to a new pose at $t = n + 1$ while perceiving a landmark, visualized here as a drone racing gate. Position and orientation of the body frame and camera image change between the time steps. Utilizing an EKF, we can fuse the sensor measurement deviations between the time steps and thus estimate the quadrotor's pose in the world frame.

such as event cameras [57] [58] and drone dynamics models [59], which aim to enhance the robustness and accuracy of VIO pipelines under high-speed conditions.

Learned Perception

All the previously mentioned drone racing teams implemented learning-based perception frameworks, either augmenting the VIO approach or implementing custom learning-based solutions.

UZH's AlphaPilot system utilized a convolutional U-net to detect gate corners in images, which were then incorporated into an EKF to correct VIO drift. Their approach aims to detect multiple gates simultaneously to build a global gate map. Additionally, their approach enabled robust gate detection even when gates were partially occluded or seen at high speed [47]. In their recent system, Swift, the perception framework is deployed as well [1].

Similarly, MavLab implemented a vision pipeline based on a U-net artificial neural network called GateNet, which produced binary masks of visible gates. Instead of relying on VIO, they used a model-based estimator that fused vision-based pose estimates with a drag-model prediction to improve robustness at high speeds [11].

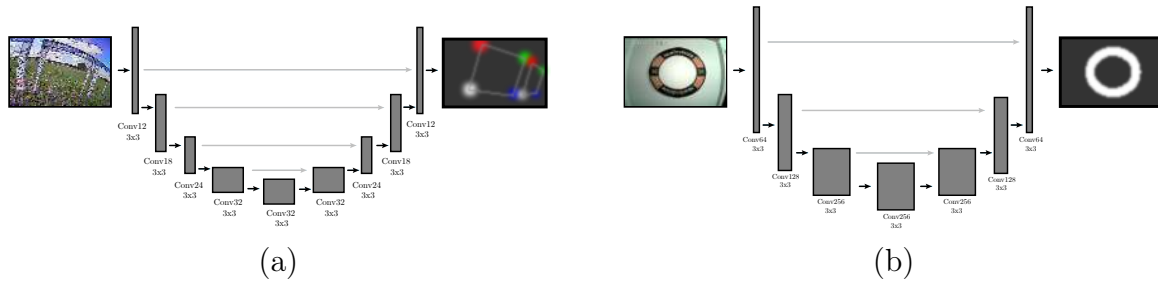


Figure 3.3: Visual Representation of the deployed perception frameworks implemented by team UZH of the University of Zurich and team MavLab of TU Delft. Both utilize a U-net convolutional neural network architecture to extract image features. (a) UZH's five-level U-net architecture. Taking an RGB image as input, the network extracts gate corners and highlights these in the feature map. (b) MavLab's four-level U-net architecture. The model is trained to predict the gate's image features on an RGB input image and outputs a binary feature map.

3.1.3 Planning

Once the drone’s state estimate is available from the perception module, the next step is to compute a trajectory that respects the drone’s dynamics and environmental constraints. The objective is to predict future drone states in a way that minimizes lap time while avoiding collisions.

The three fundamental groups of motion planning paradigms: sampling-based, search-based, and optimization-based planners have been covered in Section 2.4, where examples can be found. In the following, we introduce planners implemented in the autonomous racing frameworks by UZH and MavLab.

The UZH’s team employed a sampling-based planning approach. Their planner used a library of precomputed, dynamically feasible motion primitives to generate near time-optimal trajectories through sequences of gates [47]. The trajectory was recomputed at intervals using the current state estimate and a current global gate map. To account for onboard computation constraints, the planner used a simplified dynamics model.

TU Delft’s MavLab did not implement an explicit trajectory planning module but developed a custom solution. Their system followed a waypoint-based strategy, in which desired positions were placed at fixed distances in front of upcoming gates along their central axis [11]. The drone was commanded to fly toward these dynamically updated points using a position controller, without computing a trajectory. While this approach avoids the complexity of traditional motion planning and optimization, it relies heavily on the robustness of the state estimator. Nevertheless, it proved highly effective in practice, contributing to their victory.

UZH’s Swift policy bypasses classical planning by learning a policy that takes state observations as input and outputs control actions. The policy is trained using reinforcement learning with a reward function that encourages fast progress through the race track and keeps gates centered in the camera view [1].

As can be observed from table 3.1, each of the autonomous drone racing teams implemented a different planning approach. This leads to the assumption that there is no consensus on the optimal motion planner for the application.

3.1.4 Control

In a typical quadrotor control architecture, a high-level controller computes virtual control inputs such as body rates and collective thrust. These are then passed to a low-level flight controller that actuates the individual Motors.

Low-level open-source controllers such as PixHawk or BetaFlight are widely used in the drone racing community [14].

Model-based control utilizes the dynamics model of the system to compute control inputs that fulfill a given objective. As shown in Section 2.3, the dynamics model of a quadrotor is differentially flat. This property enables efficient trajectory generation and tracking. In [19], multiple agile maneuvers are executed on real drones using flatness-based control, which captures the system dynamics while remaining computationally efficient.

Model Predictive Control (MPC) methods are also commonly employed for quadrotor control. Due to the nonlinearity of quadrotor dynamics, nonlinear MPC is typically preferred. MPCs use the dynamics model to predict the system's states for a fixed time horizon. It then solves an optimization problem that minimizes a cost function, typically the error between setpoint and control effort. A comprehensive overview of MPC methods applied to micro aerial vehicles is provided in [60]. Nonlinear MPC is further extended in [61], where a nominal model is augmented using a Gaussian Process-based data-driven model. This approach captures aerodynamic effects that are difficult to model analytically.

Team Icarus employed a Model Predictive Path Integral (MPPI), which is a specific type of MPC controller that showed success in agile ground vehicle control [48]. MPPI control is a sampling-based optimal control method that predicts a model state similar to MPC by sampling multiple trajectories and simulating the result. These rollouts are done in parallel, and each trajectory's cost is evaluated. The costs are then used to weight the trajectories and compute an optimized control input [62].

Some methods integrate perception objectives directly into the control formulation to optimize visual information gain. These perception-aware strategies were first introduced in [63] and [64]. The goal is to follow a trajectory while keeping points of interest in view. A perception-aware MPC controller based on the quadrotor dynamics model discussed in section 2.2 was implemented in [65].

In the 2019 AlphaPilot Challenge, teams opted for traditional control strategies, which they highly customized. UZH developed a cascaded control scheme, consisting of an outer-loop proportional-derivative (PD) position controller and an inner-loop proportional (P) attitude controller. Thrust and angular velocity commands were provided at 50Hz and sent to the onboard flight controller for execution [47]. This rather simple control strategy leveraged their accurate state estimation passed from their perception and planning modules.

MavLab designed a control system that prioritized robustness and efficiency. A velocity controller was used to convert lateral and longitudinal position errors into velocity commands, which were then mapped to pitch and roll angles. Thrust was controlled using a traditional PID controller, extended with feedforward terms. Additionally, they developed a custom "pitch-for-altitude" module that compensates vertical losses during throttle saturation, and a "boost" function, which made their drone thrust immediately

on take-off before other modules had booted up. This meticulous controller formulation and especially the boost function on take-off gave the TU Delft MavLab team the edge on their competition in the 2019 Alphapilot challenge [11].

The control policy in UZH’s Swift is represented by a neural network trained in simulation via model-free RL. The network takes an observation vector composed of estimated drone states and relative gate poses as input, and outputs thrust and body rates. These commands are passed to a low-level PID flight controller implemented with the open-source Betaflight firmware. To ensure smooth transfer from simulation to the physical drone, the simulation environment is augmented with sensor noise based on real-world flight data [1].

3.2 Discussion

Across the autonomy pipeline, substantial progress has been achieved in perception, planning, and control.

In perception, visual-inertial odometry (VIO) remains a reliable state estimation framework due to its accuracy and onboard feasibility. However, VIO suffers from drift under high-speed conditions and visual degradation. To mitigate these limitations, researchers incorporate learned gate detection and fusion with VIO estimates, enabling better robustness [47]. The integration of event cameras shows promise [66], but is not widely represented in the field of autonomous drone racing due to the novelty of the system. Although several hybrid and learning-based VIO enhancements have been proposed, current methods remain sensitive to motion blur, low-texture scenes, and high dynamic range, which are typical in drone racing scenarios [67]. It is also worth noting that the winners of the 2019 AlphaPilot and 2025 AR2L, TU Delft’s MavLab team, did not include VIO in their perception approach but solely relied on learning-based perception, which raises the question of whether VIO will stay relevant for autonomous drone racing.

In planning, explicit and learned approaches are both prominent. In general, a wide variety of motion planners are implemented by autonomous drone racing teams. Employed are sampling-based motion primitive planners [47], polynomial planners [50], or custom waypoint tracking strategies [11]. Frameworks, such as Swift, realize planning entirely through learned sensorimotor policies. These learned methods require extensive simulation training and often generalize poorly to new environments. This limited adaptability arises from their reliance on training data generated in simulated environments, which restricts their ability to operate robustly under unforeseen environmental variations [14]. The investigated publications do not show a dominant planning strategy among the teams. Purely learned, but also custom hand-crafted planning paradigms show successes [1] [11].

In control, model-based methods have demonstrated precise tracking performance [19]. These approaches have been further improved with aerodynamic modeling and adaptive elements, enabling robustness to model mismatch and disturbances [47]. At the same time, learned controllers trained via RL offer end-to-end execution without reliance on internal dynamics models [68]. While these systems achieve state-of-the-art lap times, theoretical guarantees on stability and safety remain limited [14]. The importance of robust and reliable control is shown in the investigated teams’ approaches. While UZH’s learned control policy inside their Swift framework showed impressive results, most teams came up with explicit controller designs tailored to drone racing.

Overall, perception, planning, and control pipelines show good results in constrained race environments where illumination is controlled. However, open challenges remain in the areas of robustness to environmental variation, generalization to new tracks and gate types, and safety under uncertainty. Moreover, even though the learning-based methods show high performance in the selected publications, they require domain transfer techniques to bridge the sim-to-real gap.

Finally, it is worth noting that the teams' approaches are very diverse throughout all domains. Further testing and research need to be done in order to identify optimal strategies for autonomous drone racing.

4. Learning a Vision-to-Action Policy for Drone Racing

In this chapter, we will provide a detailed overview of the full hardware and software stack we employed to learn a vision-to-action policy for drone racing.

While we focus on the systems' hardware and software setup in the beginning, we cover the approach of training and implementation of our autonomy pipeline in the end.

4.1 System Setup

In the following, we introduce our hardware setup consisting of the Bitcraze Crazyflie quadrotor, which carries out all our real-world tests inside a constrained flight space. Afterwards, we establish our simulation environment and utilized open source libraries. This section serves to show what environments our experiments are conducted in.

4.1.1 Hardware Setup

As discussed in 2.1, traditional drone racing requires a race track to navigate the drone itself, FPV goggles streaming the video feed, a radio transmitter to command controls to the drone, and a human pilot to fly. For our approach to autonomous drone racing, we replace the human pilot with a computer, but the necessary components persist. Similar to market-offered quadrotors, we implemented an FPV system containing an analog camera, a video transmitter (VTX) onboard the drone, and a video receiver (VRX) connected to the static computer. Based on the received visual input, the ground station then processes the images and outputs control commands via a radio transmitter.

During all experiments, the drones' position was tracked at 100 Hz using an OptiTrack motion-capture system, while attitude angles and collective thrust commands were logged from the flight controller.

Bitcraze Crazyflie

Most autonomous drones conceptualized for racing need to adhere to computational constraints since they usually carry the central processor responsible for autonomy onboard. The smallest off-the-shelf hardware with sufficient compute for this application is the NVIDIA Jetson family. Autonomous drone racing teams usually install these computers [1] [69]. Competitions like the 2019 AlphaPilot challenge provide their standardized quadrotors with Jetsons installed. This is the main reason for the size factor of the quadrotor platforms in autonomous racing, leading to drones built on 6-inch frames [14].

Since our approach includes a ground station as a central processing platform, we are not constrained in size or weight. The choice of quadrotor was primarily based on firmware accessibility and limitations regarding spatial dimensions of the flight space. The Bitcraze Crazyflie quadrotor is a comparably small drone that weighs 27g and expands over a 92x92mm frame. While autonomous racing competitions with quadrotors of this size are currently not carried out, conventional competitions regularly host races, in which participants operate drones of up to just one inch in frame size. So-called "tiny whoops" are very popular among professional and amateur human racing pilots.

The Crazyflies' manufacturer, Bitcraze provides open-source firmware and pre-implemented state estimation modules, which make the system attractive to conduct research in domains like control or motion planning. These benefits make the system the best choice for us since we have direct access to inertial sensors and estimated states from sensor fusion.

A perceptual camera system is not provided by Bitcraze. We attached an independent FPV module containing a camera and VTX to the drone. The analog camera can perceive images at 25Hz which are then sent via the VTX over a 5.8GHz channel to be received by the VRX. The latency is estimated to be 50ms.

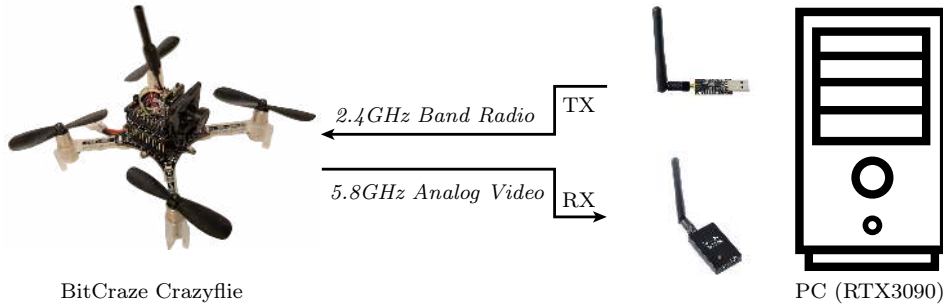


Figure 4.1: Flow diagram illustrating the hardware setup and communication channels. A picture of the Crazyflie quadrotor with an attached camera and video transmitter can be found on the left-hand side. The central PC is illustrated on the right-hand side, containing an Nvidia RTX3080 graphics card for inference. Connected to the PC via serial port are the Crazyradio transmitter and video receiver for control input transmission and image reception.

Flight Space

To conduct autonomous drone racing, we deploy policies on our customized Bitcraze Crazyflie on a race track. The race track consists of an assembly of four drone racing gates inside an enclosed flight space, which is seven meters in length, 4 meters in width, and 3 meters high. The circular gates, manufactured by Newbeedrones, span 400mm in diameter and are specifically designed for tiny whoop drone racing.

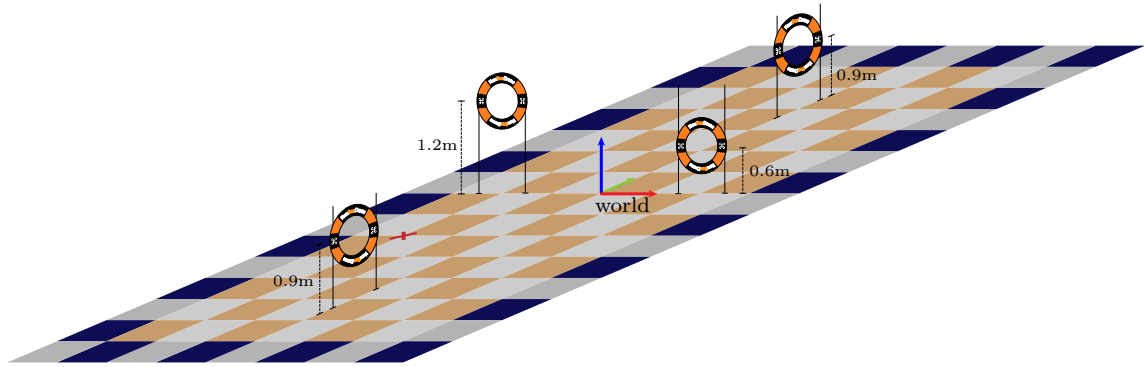


Figure 4.2: Image showing the conceived race track inside the flight space. The floor is covered with quadratic carpet tiles, each with a side length of 0.5m. The four gates are arranged to allow for circular race track navigation. A red cross indicates the start and goal position, and a Cartesian coordinate system marks the defined world frame origin.

4.1.2 Software Setup

To account for the development and learning of an autonomous policy, we develop a Python pipeline that allows us to train and deploy vision-to-action neural networks in short amounts of time in simulation using labeled synthetic images. Central modules for the realization of our Python implementation are *PyTorch*, which enables rapid research on machine learning models [70], the *bpy* python library which is also known as the stand-alone open source 3D modeling program Blender and the *Dynoplan* repository which contains the implementation of the motion planner iDbA* which is covered in section 2.4.

4.2 Approach

In the following, we go into detail on our choices for the design of an autonomous drone racing policy. While we explain our approach regarding the autonomy pipeline in the beginning, we focus on the implementation strategies of policy training and deployment afterward.



Figure 4.3: Images captured in the real flight space and the simulated flight space in Blender for comparison. The camera poses are similar, with both images taken in front of the first gate at the start of the race track. One can identify notable differences between the two images, such as color saturation, lighting, or lens distortion. (a) Image captured on the hardware while airborne at the start of the race track. (b) Simulated image inside a synthetically modeled environment in front of the race track’s initial gate.

4.2.1 Perception, Planning, and Control

With our hard and software framework set, we can now conceive the autonomy pipeline. As seen in Figure 3.1, the software modules we have to consider are *perception*, *planning*, and *control*. All of the investigated autonomous drone racing teams in 3.1.1 implemented their autonomy pipeline with a clear distinction between the three elements. We aim for a similar approach, realizing both learned perception and learned planning using neural networks. The navigation network’s output is then passed to a low-level PID controller.

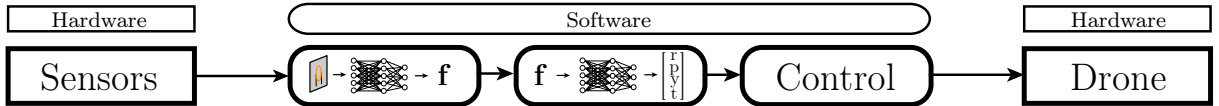


Figure 4.4: The flow diagram depicts the chosen autonomy pipeline. The architecture consists of the hardware components, sensors, and the drone. The software modules consist of our learned perception network predicting an image feature $\mathbf{f}$ and the planning policy predicting the control inputs roll, pitch, yaw rate, and thrust, which are ultimately passed to the controller. The policy predicts actions from sensed images, which are converted to motor speeds and commanded to the hardware.

4.2.2 Learned Perception

To implement our learned perception module, a convolutional neural network is implemented to estimate a feature vector $\mathbf{f} = (\mathbf{r}, \theta, \phi, \alpha, \mathbf{d}_x, \mathbf{d}_y)$ containing pose information of a racing gate based on visual input. The network accepts a 420x248px grayscale image and outputs three main components. First, the gate center’s spherical coordinates in the camera frame $(\mathbf{r}, \theta, \phi)$, second, an angle α which indicates the angle between the vector from camera to gate center $\mathbf{r}$ and the gate’s normal vector $\mathbf{n}$. Finally, a normalized two-dimensional vector $\mathbf{d}$ in the image coordinate frame, pointing in the direction from the center of the image plane to the projected center of the gate. The approach of

predicting the gate direction vector $\mathbf{d}$ is inspired by the perception network proposed in [71], which highlighted successful sim-to-real transfer. Furthermore, a 4-dimensional gate pose encoding in spherical coordinates was deployed successfully in [72].

Network Architecture

The perception model is based on a modified deep residual neural (ResNet) network [73]. The implemented ResNet-18 backbone is pre-trained on the ImageNet dataset. To match single-channel grayscale input, the first convolutional layer is replaced by a layer of equivalent dimensions, initialized with the mean of the pre-trained RGB kernels. The rest of the ResNet-18 architecture, consisting of four stages of residual block groups containing two residual blocks each, remains unchanged. The final fully connected classification layer is replaced with a linear layer shaping the feature vector into the 6-dimensional output space.

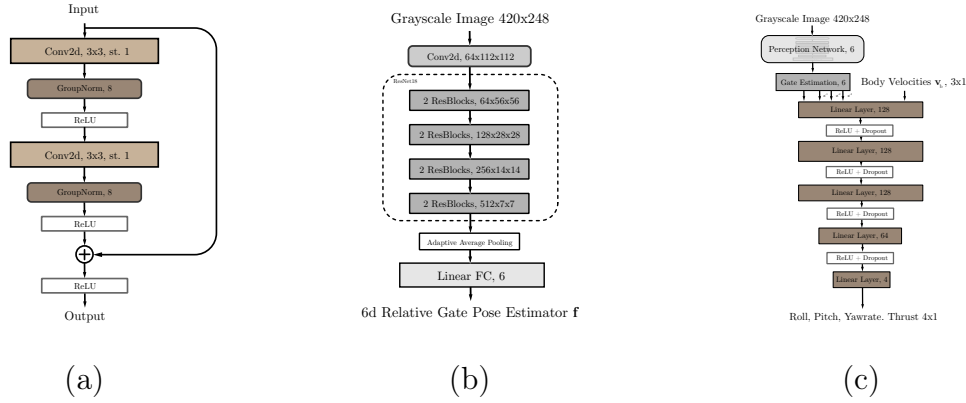


Figure 4.5: Visual Representation of our deployed neural network. While the image (c) depicts the final architecture, images (a) and (b) give insights into the perception network implementation. (a) The implemented residual block [73] with a residual connection that skips two convolutional layers. Each convolutional layer’s kernels are of shape 3-by-3 and propagate with single stride. (b) Visualization of the proposed perception net architecture. While we input a 420-by-248 pixels grayscale image, the output is represented by a 6-dimensional gate pose vector $\mathbf{f}$. The open source ResNet18, which is part of the PyTorch library, represents the backbone of our network. The provided model includes pre-trained weights inside its four groups of residual blocks, which contain two residual blocks containing two convolutional layers each. ResNets’ output is then flattened using adaptive average pooling and shaped to our 6-dimensional vector with a final linear layer. (c) The architecture of the full policy depicts the learned perception and planning module. The model extends the frozen perception module, which predicts the pose of the current gate from a grayscale image. This pose estimate, as well as the past three pose estimates, are concatenated with the drone’s current body velocity, estimated via an onboard Kalman filter. The resulting 27D vector is processed by a multi-layer perceptron (MLP) with three hidden layers, which outputs predictions for roll, pitch, yaw rate, and thrust. These predictions have been passed through appropriate activation functions to bind their ranges.

Data Generation and Labeling

Training data for the perception network is generated entirely in simulation using Blender. Each sample consists of a grayscale image of a single racing gate under randomized conditions. The camera pose is perturbed in each frame while ensuring that the gate stays within the field of view. A pool of 20 static background images gathered from the Poly Haven website is mapped onto the simulated environments’ walls, with randomized z -axis

rotations to increase background variance. Additionally, lighting conditions, Gaussian noise, and saturation are randomized during each render.

Labeling the spherical coordinates and the angle α is done as follows. Let $\mathbf{p}_{\text{cam}} = (x, y, z)^\top$ be the gate center in the camera frame (with z forward, x rightward, and y downward, as it is standard Blender convention). Samples with $z \leq 0$ (gate behind the camera) or projections outside the image are discarded. We compute

$$r = \|\mathbf{p}_{\text{cam}}\|, \quad \theta = \text{atan2}(x, z) \quad (\text{azimuth}), \quad \phi = \text{atan2}(-y, r) \quad (\text{elevation}),$$

so that the full spherical label is

$$(r, \theta, \phi).$$

Next, let $\mathbf{n}_{\text{world}} = (1, 0, 0)^\top$ be the gate's outward normal in world frame. We transform it into the camera frame via

$$\mathbf{n}_{\text{cam}} = R_{\text{cam}}^\top \mathbf{n}_{\text{world}}.$$

We also form the 3D unit-direction vector

$$\hat{\mathbf{s}} = \frac{\mathbf{p}_{\text{cam}}}{r},$$

and normalize $\mathbf{n}_{\text{cam}}$ to unit length. The angle α between the cam-to-gate direction and the gate normal is then

$$\alpha = \arccos(\mathbf{n}_{\text{cam}} \cdot \hat{\mathbf{s}}).$$

The ground truth gate direction $\mathbf{d}$ for each image is computed using the gate position in camera frame $\mathbf{p}_{\text{cam}} = (x, y, z)^\top$. The projection from the gate's world position to image coordinates is computed, and the pixel coordinates are given by the pinhole camera projection model [74]:

$$u = f_x \frac{x}{z} + c_x, \quad v = f_y \frac{y}{z} + c_y,$$

where f_x, f_y and c_x, c_y are the focal lengths and principal point offsets, that have been computed using the OpenCV library. The pixel coordinates (u, v) of the projected gate center are then normalized to obtain the gate direction vector:

$$\mathbf{d} = \left(\frac{u - W/2}{W/2}, \frac{v - H/2}{H/2} \right),$$

where W and H denote the width and height of the image.

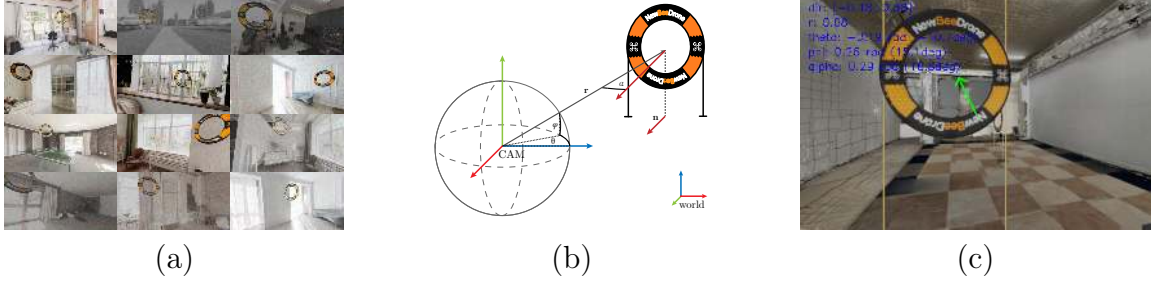


Figure 4.6: Three images representing training data and labeling process for our proposed perception net. While the images are converted to grayscale, they are represented with RGB information here for better visibility. (a) Composition of twelve example images from the network’s training dataset. The full dataset consists of 20000 images of the gate taken from random camera poses in front of a gate, with the gate always being in frame. Additionally, background image, lightning, and noise are also randomized for each of the training images. Each image is labeled with the gate pose vector $\mathbf{f}$. (b) Image indicating the spatial properties of our proposed label. The camera frame with its spherical coordinate system is shown on the left-hand side, while the gate in camera frame is represented on the right. Observed can be the spatial labels $(\mathbf{r}, \theta, \phi, \alpha)$. (c) Labeled simulated image containing a racing gate. While the green arrow represents the gate direction vector $\mathbf{d}$, the blue text in the top left of the image shows the labeled values of the six-dimensional gate pose vector $\mathbf{f}$.

4.2.3 Learned Planning

To simplify motion planning and enable quadrotor navigation based on images and estimated body velocities, we implement a learned policy that predicts low-level control commands. The proposed model builds on top of the previously introduced perception network by extending it with a control head.

In addition to the predicted gate pose vector and its three priors $\hat{\mathbf{f}}_t, \hat{\mathbf{f}}_{t-1}, \hat{\mathbf{f}}_{t-2}, \hat{\mathbf{f}}_{t-3}$, the network also intakes the current body-frame velocity $\mathbf{v}_b$ of the drone, estimated by its onboard EKF. The combined input is a 27-dimensional vector composed of the predicted gate poses $(\hat{\mathbf{r}}_{t:t-3}, \hat{\theta}_{t:t-3}, \hat{\phi}_{t:t-3}, \hat{\alpha}_{t:t-3}, \hat{\mathbf{d}}_{\mathbf{x},t:t-3}, \hat{\mathbf{d}}_{\mathbf{y},t:t-3})$ and the velocity vector (v_x, v_y, v_z) . Due to buffering the spatial information in $\hat{\mathbf{f}}$, we enable the policy to infer the quadrotors’ motion.

These inputs are processed by a multi-layer perceptron which consists of three hidden layers of size 128, 128, and 64, followed by a final linear layer producing three outputs as seen in figure 4.5 (c). These correspond to desired roll, pitch, yaw rate, and thrust.

Training of this model is realized with the imitation learning paradigm DAgger [7], which will be discussed in detail in section 4.2.4.

Low-Level Controller

The low-level control stack onboard the drone, provided by BitCraze, implements a cascaded PID architecture that regulates the drone’s rotational dynamics and thrust [25]. The control structure is designed to support position, velocity, or attitude commands. In our case, the neural network outputs commands in the form of roll, pitch, yaw rate, and thrust (RPYT), which are then forwarded to the control stack.

Roll and pitch angles are tracked using an attitude controller, which computes desired angular rates and minimizes the attitude deviation with respect to the current orientation that is estimated by the EKF. These angular rate setpoints, as well as the yaw rate, are then regulated by an inner-loop attitude-rate controller. The controller runs at 500 Hz.

The thrust and the three body torques are combined into a four-dimensional control vector, which is passed through a motor mixing matrix that accounts for the quadrotor’s geometry and rotor orientations. The resulting four motor thrust commands are then mapped to PWM duty cycles using a nonlinear mapping and sent to the motors.

This low-level control infrastructure enables real-time execution of RPYT commands generated by the neural network and provides closed-loop stabilization of the drone.

Additionally, state estimation is performed onboard using an EKF that fuses IMU data. When available, the Bitcraze Flow Deck V2 provides additional sensor modalities, including a downward-facing optical flow sensor and a time-of-flight rangefinder. These allow for more accurate velocity and altitude estimation. This velocity estimate represents an input $\mathbf{v}_b$ to our proposed planner policy in section 4.2.3.

4.2.4 Implementation

In the following, we explain the implementation of our training pipeline for the conceived policy, while also covering the algorithm enabling the policy to command the hardware. Due to the controller already being implemented on the quadrotors’ flight controller by the manufacturer, it will not be covered.

Dagger

As introduced in section 2.5, DAgger is an imitation learning algorithm that addresses distributional shift, a common issue in classical supervised learning. By collecting data under the student policy’s distribution and relabeling with expert supervision, the mismatch between states encountered during training and states encountered during testing is equalized.

We apply DAgger to learn a vision-based motion planning policy for autonomous drone racing. As an expert policy, we use the motion planner iDbA*, introduced in section 2.4. Provided with the robot’s dynamics, the environment, and start and goal states, iDbA* computes a locally optimal trajectory from start to goal. The algorithm accurately portrays the desired behavior for drone racing.

To implement DAgger for the special case of training a vision-to-action autonomous drone racing policy, we modified algorithm 3 towards our needs. Our customized version of DAgger, which is deployed during policy training, can be found in algorithm 4. The main difference between DAgger’s base implementation in algorithm 3 is that we limit the student policy’s exploration capabilities. A rollout will terminate once the drone’s current state exceeds the environment’s bounds. Furthermore, we provide additional practical information on state propagation, the decay of β , and the overall script structure.

We now provide a comprehensive description of Algorithm 4, explaining the underlying principles and thought process that guided its implementation.

Initially, the necessary variables and a random policy are initialized. In addition to that, we load a Blender environment and start the rendering engine to account for synthetic image generation.

Starting from line 5, an outer loop, which iterates over the desired number of DAgger iterations, captures an initial I_0 image from the simulation at the current state $\mathbf{s}$.

An inner while loop in line 8 will generate a trajectory, executing a single step for each iteration, as long as the trajectory consisting of states $\mathbf{s}$ has not left the set environment $\mathcal{E}$ and has not reached the goal state $\mathbf{s}_g$. Based on the mixing coefficient β , an if condition inside the while loop determines whether the step to be executed is based on a prediction by the current policy $\hat{\pi}_i$ or a computation by the expert policy π^* , which is the iDbA* algorithm. Prior to updating state and image for the next iteration, the current image and action pair $(I, \mathbf{a})$ is added to the dataset $\mathcal{D}$. These rollouts will consist mostly of expert states $\tau_k^*[0]$ for early DAgger iterations, while later ones will only consist of states predicted by the policy $\hat{\pi}_i(I)$, since β decays over time.

An inner for loop in line 17 iterates over all states that were just encountered during the rollout. Here, iDbA* is queried to receive the expert action $\tau_k^*[0]$ for each state. The images I_k in the dataset $\mathcal{D}$ are then relabeled, now being assigned to expert actions. This step is crucial for the functionality of DAgger, taking advantage of the student policy’s exploration during the rollout and thus covering the state space while keeping every state consistent with the expert policy’s behavior due to the relabeling.

Ultimately, the DAgger iteration concludes by training the next policy $\hat{\pi}_{i+1}$ on the full dataset $\mathcal{D}$, updating the mixing coefficient β and resetting the current state $\mathbf{s}$.

Algorithm 4: DAgger for a Vision-Based Autonomous Drone Racing Policy

Input: Environment $\mathcal{E}$, dynamics f , start state $\mathbf{s}_0$, goal $\mathbf{s}_g$, expert planner iDbA*, iterations N , decay function $\text{decay}(\cdot)$

Output: Learned policy $\hat{\pi}$ with best performance

```

1  $\mathcal{D} \leftarrow \emptyset$ 
2  $\hat{\pi}_1 \leftarrow$  initial policy
3  $\beta \leftarrow 1.0$ 
4  $\mathbf{s} \leftarrow \mathbf{s}_0$ 
5 Initialize rendering engine
6 for  $i = 1$  to  $N$  do
7   Capture image  $I_0$  at  $\mathbf{s}$ 
8   while  $\mathbf{s} \neq \mathbf{s}_g \wedge \mathbf{s} \in \mathcal{E}$  do
9     if  $u \sim \mathcal{U}(0, 1) < \beta$  then
10       Compute expert traj.  $\tau^* \leftarrow \text{iDbA}^*(\mathbf{s}, \mathbf{s}_g, \mathcal{E}, f)$ 
11       Set  $\mathbf{a} \leftarrow \tau^*[0]$ 
12     else
13       Predict  $\mathbf{a} \leftarrow \hat{\pi}_i(I)$ 
14       Aggregate  $\mathcal{D} \leftarrow \mathcal{D} \cup \{(I, \mathbf{a})\}$ 
15       Propagate state  $\mathbf{s} \leftarrow f(f(\mathbf{s}, \mathbf{a}), \mathbf{a})$ 
16       Capture new image  $I$  at  $\mathbf{s}$ 
17   foreach  $(I_k, s_k) \in \mathcal{D}$  do
18     Query expert  $\tau_k^* \leftarrow \text{iDbA}^*(\mathbf{s}_k, \mathbf{s}_g, \mathcal{E}, f)$ 
19     Relabel  $\mathbf{a}_k \leftarrow \tau_k^*[0]$ 
20   Train  $\hat{\pi}_{i+1}$  on  $\mathcal{D}$ 
21   Update  $\beta \leftarrow \text{decay}(\beta)$ 
22   Reset  $\mathbf{s} \leftarrow \mathbf{s}_0$ 
23 return  $\hat{\pi}_i$  with best performance

```

Policy Deployment

Once a promising policy is trained using DAgger, we can deploy it on the hardware. To realize the required functionalities, we implement a Python script, which receives images from the camera attached to the quadrotor. The script then predicts an action based on the image and the logged velocities, utilizing our trained policy. Finally, the predicted roll, pitch, yaw rate, and thrust control inputs are commanded to the quadrotor, concluding the script's main loop.

Polling of the images, predicting, and commanding needs to take place in limited time, since camera images are received at 25Hz and smooth navigation of the Crazyflie is desired. The chosen hardware setup, including a base computer containing an Nvidia RTX3080 graphics card, accounts for that. Using this setup, we can reliably predict on every image received.

Algorithm 5: Drone Racing Policy Deployment on a Bitcraze Crazyflie Quadrotor

```

1 Start video stream  $V$ 
2 Connect to Crazyflie  $cf$ 
3 Assign trained policy  $\pi$ 
4 Set time interval  $dt$ 
5 while  $cf$  is armed do
6   Log current time  $t_{start}$ 
7   Get latest image  $I_t$  from video stream  $V$ 
8   Read current body velocities  $\mathbf{v}_b$ 
9   Deploy policy [roll, pitch, yaw rate, thrust]  $\leftarrow \pi(I_t, \mathbf{v}_b)$ 
10  Send setpoint to Crazyflie  $cf$ (roll, pitch, yaw rate, thrust)
11  Log current time  $t_{end}$ 
12  Calculate elapsed time  $t_{elapsed} = t_{end} - t_{start}$ 
13  Sleep for  $\max(0, dt - t_{elapsed})$ 

```

5. Experiments

In this chapter, we will deploy two selected developed policies in simulation and on real hardware. Under fixed conditions, we will compare the policy’s performances along multiple metrics with the performance of a human pilot. In addition to policy tests, we also document the process that led to the final architecture of our policy, highlighting advantages and disadvantages along the way.

5.1 Developmental Iterations and Failures

The following sections provide a comprehensive overview of the iterative experimental process that was carried out during the derivation of our autonomous drone racing policy.

While we planned to train an End-to-End policy initially, we found that this type of autonomy pipeline does not transfer from simulation to hardware. To account for the sim-to-real gap between synthetic and camera images, we implemented different types of image transformations, such as a pencil filter [67].

Since preprocessing of the images did not close the perception gap between simulation and reality, we eventually opted for the extension of our network, now consisting of two separate perception and planning policies. In this stage, the perception network predicted just a 2-dimensional gate direction vector, which was fed into a 2-layer MLP.

During the final iteration, we aimed to refine the introduced perception backbone network and extend the navigation network, reaching the architecture shown in Figure 4.5.

We will now go into detail on the policy progression stages. Note that throughout all of these stages, our hardware setup remained consistent, but the policy was perturbed.

5.1.1 End-to-End Policy

As an initial approach, we implemented a policy predicting actions directly from 420x248px RGB images and the drone's body velocities. The network was trained end-to-end in simulation using DAgger.

The network architecture consists of three convolutional layers, to which the RGB image was fed. The output was concatenated with the three body velocities and fed into a max-pooling layer that was flattened and fully connected to predict the three-dimensional action vector. In contrast to our current policy introduced in section 4.2.3, the convolutional layer's parameters were not frozen during training, as there is no clear distinction between perception and planning in this approach.

After 82 iterations of DAgger training and 15000 generated synthetic images, the policy was successfully navigating the race track in simulation.

When deploying the policy on our hardware, we found that the quadrotor was not able to navigate the parkour. Furthermore, the policy's behavior was inconsistent, taking different paths for each conducted test flight. While for some trials the policy managed to navigate roughly in the correct direction of the first gate, others were turning around or drifting off. We detail this hardware test further in section 5.3.3.

As a parameter that is fed to the network, we identified the main issue to be the difference between synthetic and camera images. To that end, we implemented a variety of image transformations, which did not affect the policy's performance on the hardware.

We found that training a neural network to predict low-level quadrotor control inputs for drone racing from raw images likely requires large amounts of real data, which is difficult to obtain. This is also suggested by findings of other researchers [71].

Additionally, we identified that a major shortcoming of an end-to-end perception and navigation policy is the ability to debug the cause of failure.

5.1.2 Pencil-Filter Image Processing

To close the sim-to-real gap regarding images, we opted to further preprocess both simulated and camera images so that our policy would generalize better to relevant image features and thus ease its transfer to the hardware.

A recent publication by researchers of the AirLab at Aarhus University proposed a novel method for drone racing gates perception [67]. A convolutional neural network is introduced that allows for zero-shot sim-to-real transfer learning for robust gate perception in autonomous drone racing. The researchers state that by transforming images using a low-cost morphological operation, called pencil filtering, they bridge the gap between simulation and reality, which allows them to train a robust perception policy solely on simulated images.

Algorithm 6: Pencil Filter (Simplified)

Input: RGB image I
Output: Filtered pencil image P

- 1 Convert I to grayscale image G
- 2 Apply morphological dilation to G using an elliptical kernel to obtain P
- 3 **foreach** pixel row index y in P **do**
- 4 **foreach** pixel column index x in P **do**
- 5 **if** $P(y, x) = 0$ **then**
- 6 Set $P(y, x) \leftarrow 255$
- 7 **else**
- 8 Set $P(y, x) \leftarrow \text{int} \left(255 \cdot \frac{G(y, x)}{P(y, x)} \right)$
- 9 **return** P

We adopted the pencil filter from [67] to preprocess every image fed to our end-to-end policy for both synthetic and camera images. While the policy showed good results in simulation, navigating the track after 17000 images taken during DAgger training, it was not able to transfer to the real world and did not show improved performance in comparison to the prior policy, which had not utilized the pencil filter.

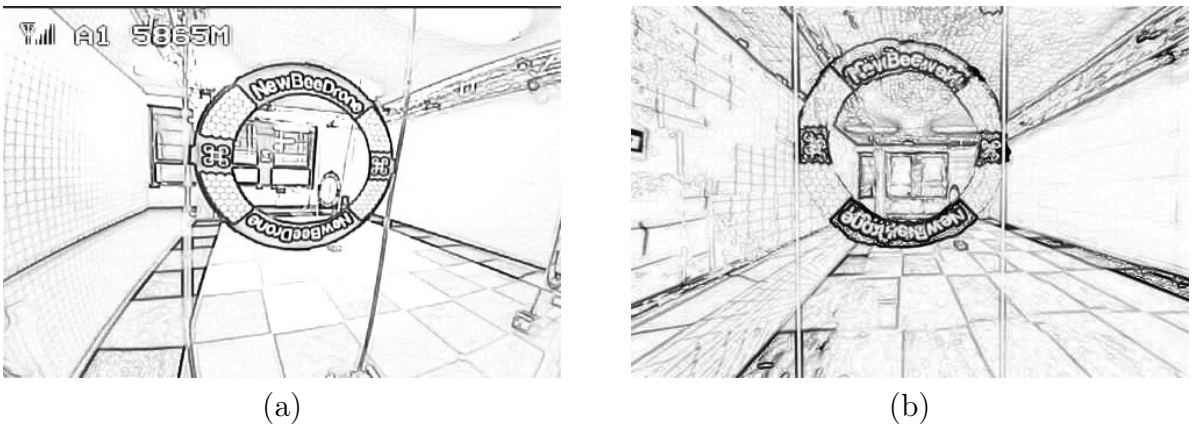


Figure 5.1: Two images depicting the resulting output of the pencil filter for a simulated and a camera image. (a) Camera image captured on the real race track in front of the first gate, transformed by the pencil filter. (b) Simulated image after pencil filtering taken at a similar pose inside the Blender environment.

While the transformed images appeared visibly more alike than their RGB equivalents in figure 4.3, we came to the conclusion that pencil filtering would not be sufficient to close the sim-to-real gap for a policy that is trained end-to-end and decided to adjust the policy’s architecture.

5.1.3 Gate Direction Perception

As image transformations did not suffice to make the policy transfer from training in simulation to deployment on hardware, we decided to change the policy’s architecture. The main motivation behind this was the inability to debug our convolutional neural network. Since the network predicts actions directly from images, we had no indicator to verify whether the policy would recognize a gate or not. The only observation to make was if the predicted actions allowed for track navigation by the Crazyflie, which they did not.

Based on autonomous drone racing pipelines in literature [1], [71], [11], we implemented two separate perception and planning policies, similar to our final proposed architecture in section 4.2.2. This prior version of the perception policy consisted of a pre-trained ResNet18 backbone and predicted a normalized 2-dimensional gate direction vector that points from the image center towards the projected center of the next racing gate. This output was then concatenated with the quadrotors’ 3-dimensional body velocities and fed to a two-hidden-layer MLP that consisted of a 64-neuron fully connected (FC) layer and another 32-neuron FC layer before outputting three actions.

The policy was trained in simulation, with the already trained perception network being frozen during DAgger training. The policy was unable to navigate the race track in simulation after 10000 images were created from 50 DAgger iterations. Training this architecture for a higher number of DAgger iterations would likely enable the policy to navigate the complete race track. Regardless of policy performance, we were now able to indicate whether the policy recognized the racing gate by evaluating the predicted gate direction vector. The perception policy could robustly predict gate center points when fed simulated images.

To clarify if the sim-to-real gap would be persistent, we carried out a test, predicting the gate direction vector from real camera input images of racing gates. We found that the perception network robustly predicted valid directions for the racing gates’ center points, thus closing the gap between simulation and reality, which is further detailed in section 5.3.1.

This approach was then extended by adding the network outputs $\mathbf{r}, \theta, \phi, \alpha$. As shown in Figure 4.6, these labels indicate the pose of the gate’s center in spherical coordinates inside the camera frame. Furthermore, the final planning policy from section 4.2.3 predicts roll, pitch, yaw rate, and thrust (RPYT) instead of three accelerations along world axes. This proves beneficial, since RPYT’s reference system is the drone’s body frame, whereas accelerations in world frame change signs in x and y during circular race track navigation.

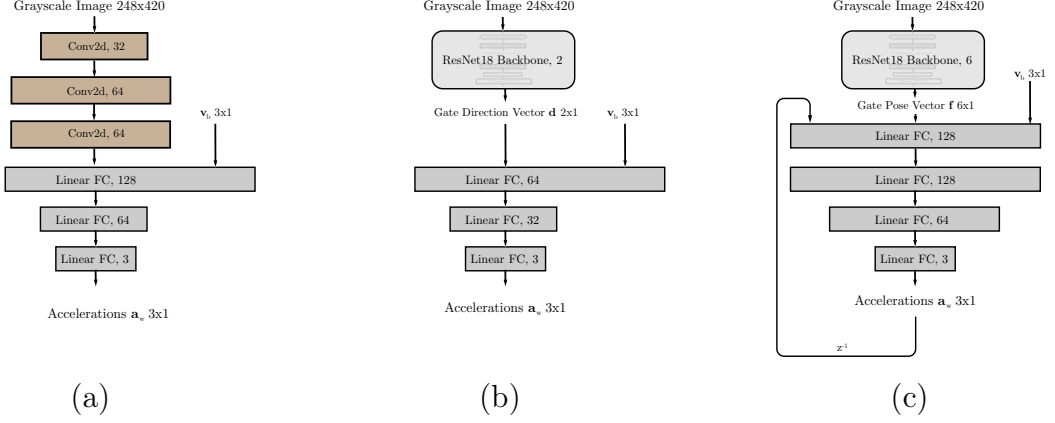


Figure 5.2: Three images illustrate different stages of policy architectures that were deployed throughout the experimental process. (a) End-to-end trained policy consisting of three convolutional and three fully connected linear layers. Predicting three accelerations from a camera image and the quadrotors’ body velocities. (b) Policy architecture consisting of a ResNet18 backbone predicting a two-dimensional gate direction vector, which is passed to a three-layer MLP alongside the drone’s three body velocities. The network’s output is three accelerations in world frame. (c) Neural network predicting three accelerations from image and velocity inputs. The image is fed through a perception backbone, which predicts a six-dimensional gate pose vector, which is concatenated with the three-dimensional body velocities and passed through a four-layer MLP. Additionally, the predicted action is fed back as input to the MLP at the next time step.

Figure 5.2 illustrates our discussed discarded policy architectures. Each of the three images represents a development stage that culminated in our final approach represented in 4.5, with the main differences being MLP size, perception net output size, and the provision of the past three predicted gate poses to the MLP.

As seen in figure 5.2 (c), we provided a memory of the networks’ past predicted actions as well. This has proven to enhance the smoothness of the control outputs as a recent study published by the University of Zurich’s Robotics and Perception Group highlighted [75]. However, it did not improve navigation performance in simulation.

5.2 Experiment Metrics and Evaluation Criteria

To evaluate both the speed and robustness of our autonomous policies relative to a human expert baseline, we analyze four metrics.

First, we compare the traveled distance per lap d_{lap} , as it is a measure of racing efficiency. Shorter lap distances usually correspond to more direct, optimal racing lines [13].

Second, speed performance is characterized by per-lap completion times T_i , from which we compute the mean lap time

$$\bar{T} = \frac{1}{M} \sum_{i=1}^M T_i$$

and the best lap time $T_{min} = \min_i T_i$, where M is the total number of laps flown.

Third, robustness is quantified via the crash rate

$$C = \frac{N_{\text{crashes}}}{M},$$

with N_{crashes} denoting the total number of collisions with gates or course boundaries.

Finally, to measure the sim-to-real transfer gap in terms of gates an agent passed before crashing, we define a crash-per-gate rate. Let G_i be the number of gates successfully passed in trial i . Then:

$$C_g = \frac{N_{\text{crashes}}}{\sum_{i=1}^M G_i}.$$

The sim-to-real transfer gaps are

$$\Delta C = C_{\text{real}} - C_{\text{sim}}, \quad \Delta C_g = C_{g,\text{real}} - C_{g,\text{sim}}.$$

Additionally, we provide an evaluation in regards to the perception policy’s accuracy.

5.3 Evaluation

Each experiment’s result will be presented in this section. While we share findings regarding the transferability of the implemented perception module in the beginning, we will give detailed reports on the flight tests afterward and finally evaluate the conducted experiments quantitatively in regard to our proposed evaluation metrics.

We deployed two different racing policies that showed promising performance in simulation. The selected trained neural network architectures are represented by the sparse approach implemented in section 5.1.1, which was trained end-to-end, and our fully developed racing policy from section 4.2.3 with separate perception and planning modules.

5.3.1 Perception Policy

To evaluate the trained perception network, we provide details on the training process, as well as testing perception performance on real gate images, indicating the transferability from sim-to-real.

We trained the model on a dataset of 20000 simulated, single-channel gate images labeled with the 6-dimensional gate pose vector $\mathbf{f}$. The data was split into 80% training and 20% testing subsets. The training was performed for up to 100 epochs on a GPU using the Adam optimizer with a *ReduceLROnPlateau* learning rate scheduler. We employed mean-squared error as the loss function between the predicted direction vector $\hat{\mathbf{f}}$ and the ground truth vector $\mathbf{f}$:

$$\mathcal{L} = \left\| \hat{\mathbf{f}} - \mathbf{f} \right\|_2^2.$$

The model weights corresponding to the lowest test loss were saved.



Figure 5.3: Two images implying the perception nets' performance. We show its prediction capabilities on a simulated and a camera image. Since both images were taken at similar camera poses, one can validate the network's ability to transfer from simulation to reality. (a) Simulated image of the race track taken in front of the second gate. The predicted gate pose vector is indicated in the top left corner, with the gate direction vector $\mathbf{d}$ represented by a red arrow pointing from the image center towards the gate center. (b) Camera image proving the perception nets' transferability from simulation to reality. The policy's predictions are printed in the top left corner of the image, which was taken at a position prior to passing the second gate of the race track.

The model's predicted gate pose vectors $\hat{\mathbf{f}}$ show similar accuracy when comparing synthetic and real images, as seen in figure 5.3 (a) and (b). While camera position and orientation as well as gate orientation slightly differ between the images, the predicted values for angles $(\hat{\theta}, \hat{\phi}, \hat{\alpha})$, gate direction $\hat{\mathbf{d}}$ and gate distance $\hat{r}$ are consistent with the gates pose when observed visually. Furthermore, this perception performance proves robust after predicting on multiple images.

5.3.2 Human Piloted Flight

To establish a baseline for our policy comparison, we conducted human-piloted flights on our proposed race track. The pilot was granted a practice period to become acquainted with the course layout. During the recorded sessions, the human-controlled quadrotor was released in front of the first racing gate at position $\mathbf{p}_{\text{start}} = [-1, -1.5, 1]$.

To enhance the comparability of this manual flight to our expert policy, we constrained the CrazyFlies' maximum angular rates and thrust to:

$$\begin{aligned}\dot{\phi}_{\max} &= 60^\circ/\text{s}, \\ \dot{\theta}_{\max} &= 60^\circ/\text{s}, \\ \dot{\psi}_{\max} &= 100^\circ/\text{s}, \\ T_{\max} &= 65\%.\end{aligned}$$

This is necessary since iDbA*'s motion primitives adhere to a range of viable states and actions. By constraining our hardware to stay inside these ranges, we create similar conditions and allow for a direct comparison between the iDbA* and the human pilot.

The pilot flew consecutive laps until the battery ran out or they crashed. In total, we recorded 20 laps over 5 batteries. While the pilot successfully managed to navigate the race during batteries three and four, the trials during batteries one, two, and five were stopped by crashes.

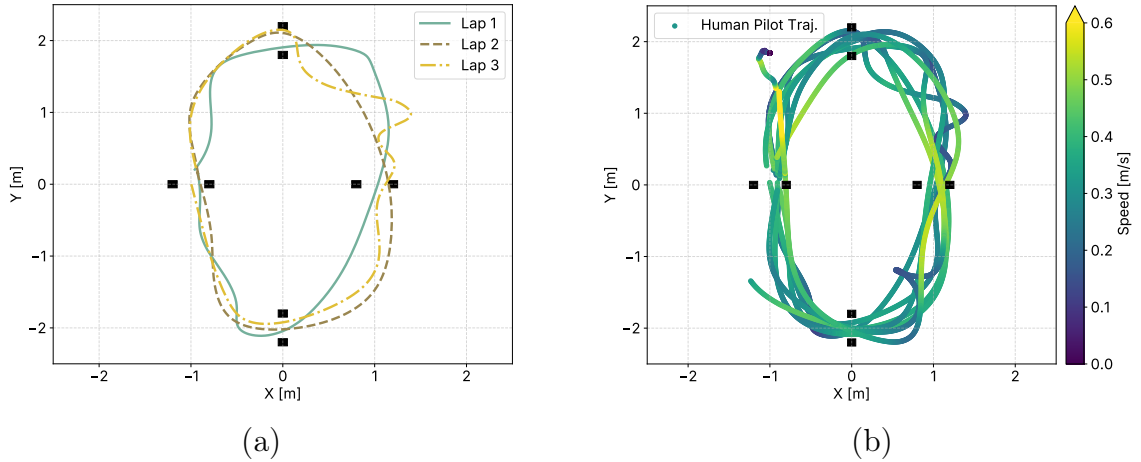


Figure 5.4: Two images providing information on the conducted human-piloted quadrotor flights. (a) Two-dimensional projection of the trajectory paths of three selected consecutive laps on our race track. The racing gates' positions are represented by two black boxes per gate. Each Lap is indicated by a different line style to illustrate the variety of flown trajectories in regard to smoothness and traveled distance. (b) Quadrotors' projected x-y position during a human-piloted flight. A color bar indicates the drone's velocity at each position. The recorded trajectory shows seven laps.

Figure 5.4 (a) depicts the two-dimensional projection for three representative race track laps in the x-y plane. Lap 3 is an example in which the pilot failed to align the quadrotor with the gates' normal $\mathbf{n}$, resulting in a pronounced correction. This behavior is particularly visible before passing gate three at $[1, 0]$. In contrast, Lap 2 illustrates one of the fastest trajectories. It is characterized by continuous gate passages without correcting and an

overall smooth profile. Lap 1 represents a stable trajectory with only a minor corrective adjustment at the gate located at $[-1,0]$. Lap 1 corresponds to the median lap performance.

Figure 5.4 (b) shows the complete trajectory for the fifth battery cycle, consisting of six successful laps followed by a crash at position $\mathbf{p}_{\text{crash}} = [-1, 1.8]$. In the plot, the velocity for each state is encoded by a continuous color scale. Peak velocities occur along the straight segments at $x = -1$ and $x = 1$, while substantial velocity drops coincide with orientation mismatches between the drone and gate where corrective braking was required. Deceleration during turning maneuvers is also evident, which is expected given the increased centripetal forces.

5.3.3 Autonomous Drone Racing Policy

We now cover the performance of our two selected trained policies. As a reference for their performance, the expert simulation and human-piloted flight trajectories are taken into account.

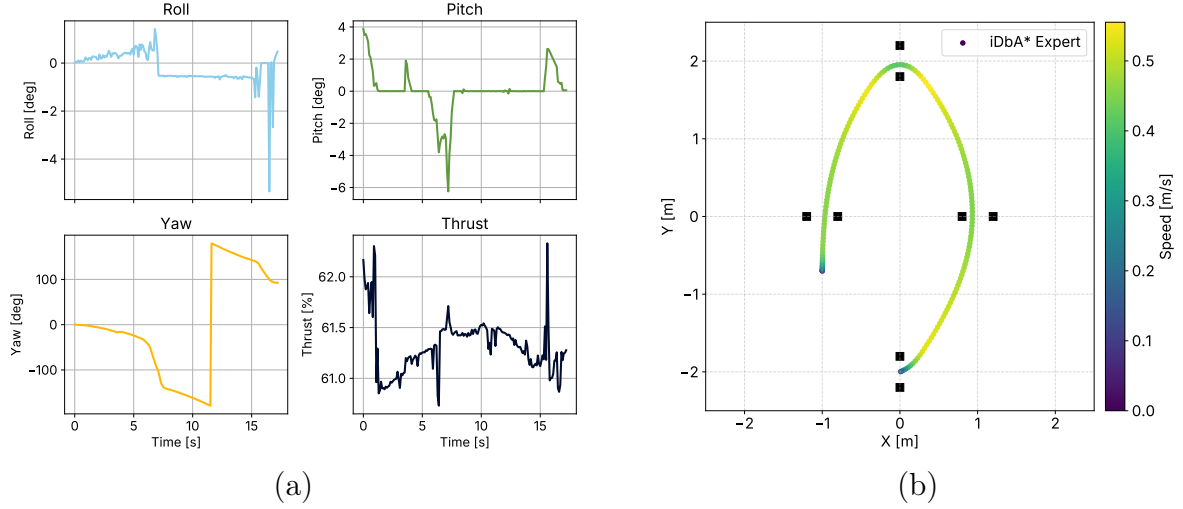


Figure 5.5: Navigation performance of the iDbA* planner on the proposed race track in simulation. (a) Time histories of quadrotor roll, pitch, and yaw angles, together with the thrust profile. (b) Planned trajectory in the horizontal plane from start at $\mathbf{p}_{\text{start}} = [-1, -0.8, 1]$ to goal at $\mathbf{p}_{\text{goal}} = [0, -2, 1]$. Black squares denote racing gate locations, and the trajectory is color-coded by each state’s current velocity.

Figure 5.5 represents iDbA*’s navigation characteristics in simulation provided our double integrator dynamics model (section 2.2), the race track environment (represented in Figure 4.2), start and goal state, and a set of motion primitives.

The computed trajectory in Figure 5.5 (b) shows smooth transitions between states, regarding both position and velocity. At each of the turns during a gate passage, iDbA* keeps its path close to the boundaries of each gate, located on the inside of the turn. This further highlights the planner’s local optimality.

The double integrators’ speed never exceeds a maximum velocity of 0.6m/s. This is due to the set of available motion primitives spreading over a range of -0.6m/s to 0.6m/s. Thus, iDbA* exerts time optimality in regards to the predefined set of motion primitives [17].

Corresponding time histories of commanded roll, pitch, and yaw angles as well as the collective thrust are visualized in Figure 5.5 (a). While the yaw angles' monotony indicates smooth turns, the computed roll, pitch, and thrust axes have oscillatory tendencies. Initial acceleration at the start state is provided by high levels in pitch and thrust, which drop off after maximum velocity is reached. This style of behavior continues throughout the action histories, raising pitch and thrust to accelerate, lowering to decelerate. The roll axis's contribution is mainly to lean inwards the circular track, exerting its highest action on approaching the goal state.

Trained End-to-End

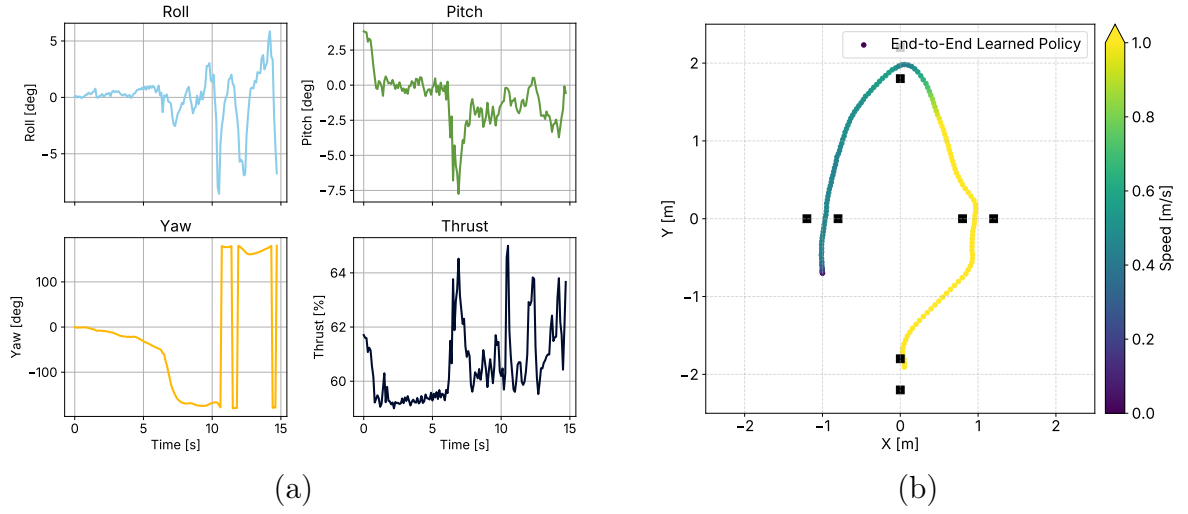


Figure 5.6: Two plots depicting navigation characteristics of our end-to-end trained policy in simulation. (a) Predicted roll, pitch, yaw, and thrust values over time. (b) Projected trajectory in the x-y plane from start at $p_{\text{start}} = [-1, -0.8, 1]$ to goal at $p_{\text{goal}} = [0, -2, 1]$. The racing gates' locations are represented by black squares. Each state's current velocity is indicated by a color coding.

Figure 5.6 (a) shows the attitude and thrust time histories predicted by the end-to-end trained, imitation-learned policy after 80 DAgger iterations (15000 labeled images generated). In the initial segment (0–3s), pitch remains below the expert reference profile from figure 5.5, resulting in velocities that are insufficient to keep up with the expert. This is indicated further when comparing drone speeds at the start of each test in figures 5.6 (b) and 5.5 (b). During the navigation of the first corner ($t \approx 7\text{s}$), roll and yaw trajectories conform to the expert reference with minor differences in amplitude. As the quadrotor approaches the third gate, which is placed at a lower height, as seen in 4.2, pitch is reduced. Here, the thrust command predicted by the trained policy is well above the expert action for this state. This results in an accelerated descent and an overshoot in speed that keeps increasing until reaching the goal state. This velocity mismatch between expert and trained policy during the second half of race track navigation is visible when comparing figures 5.6 (b) and 5.5 (b). In the final segment (12–15s), the yaw signal exhibits oscillations that are not present in the expert data. These discrepancies are attributed to the sparse sampling of states in the goal region. DAgger rollouts begin at our fixed start state and terminate on reaching a maximum amount of sampled states or when in collision, resulting in few training samples near the target position. Extending the number of DAgger iterations or introducing randomized start positions along the expert path would improve coverage of

states at the end of the race track and thus improve policy performance in this sector.

Despite the mentioned discrepancies, our sparse, end-to-end trained policy successfully navigates the complete race track in simulation.

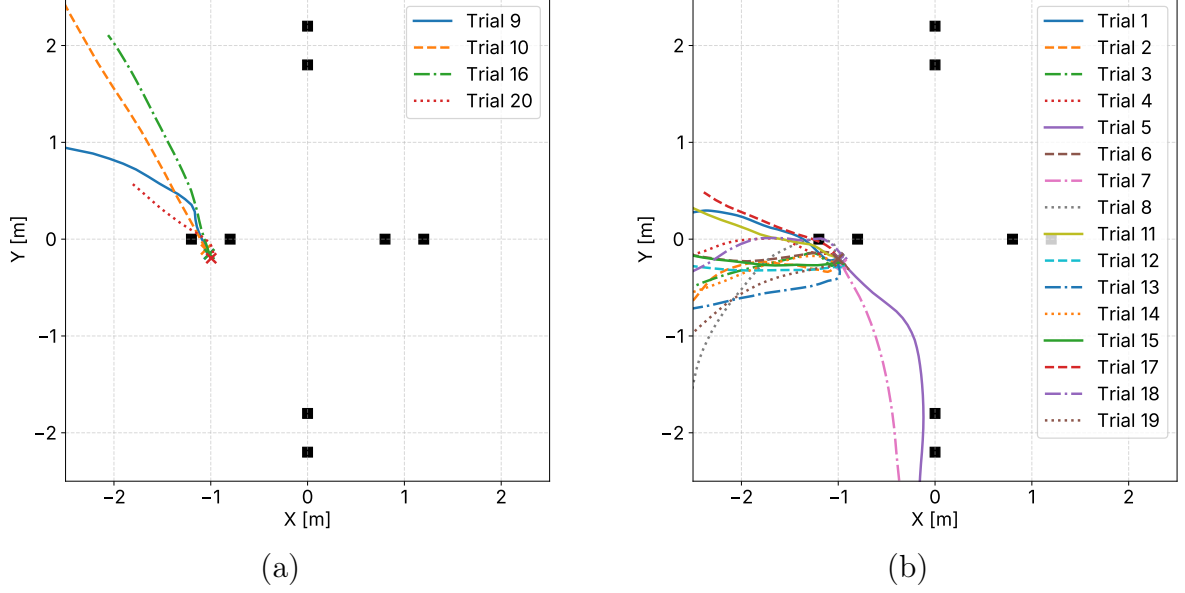


Figure 5.7: Visual representation of the end-to-end trained policy’s flight performance on the real-world race track after 20 test flights. Each trajectory can be identified by line style and corresponding trial index. Flights’ starting positions are indicated by crosses in the trajectories’ respective colors. (a) Four out of the 20 conducted flights are represented as projected trajectories in the x-y plane. After deploying the policy from the starting position at $[-1, 0.2, 1]$, navigation through the first racing gate was feasible. However, subsequent to the first gate passage, none of the recorded trajectories manages to stay on track and eventually crashes. Racing gates are indicated by black squares. (b) Plot depicting the horizontal projection in the x-y plane of the 16 remaining trials. None of the paths taken by the end-to-end trained policy managed to pass the first gate, with all of them ending in a crash.

Figure 5.7 shows the policy’s performance when deployed on hardware inside the flight space. Out of 20 conducted test flights, we recorded four trajectories with the Crazyflie managing to pass the first gate of the race track, which is indicated in figure 5.7 (a). Among these four trials, our policy would not be able to navigate towards a consecutive gate, which led to all experiments terminating prior to reaching the desired goal state.

When putting these four trials into perspective with the remaining 16 carried out flights in figure 5.7, in which the drone would not pass any gate. We find that even a singular gate passage can not be executed robustly by the end-to-end trained policy. Overall, the trajectories exhibit a consistent leftward turn (except for trials 17 and 5) immediately after takeoff. Since the policy never executed a left turn after takeoff in simulation, this behavior implies that the states encountered during hardware deployment lie outside the simulated state distribution. This distributional shift strongly suggests that the policy has overfitted to the simulation environment.

We found that approaching the first gate at a slight angle towards the gate’s right side, so that the camera would have more exposure of the inside of the race track, would increase the chances of passing the first gate.

Trained with Frozen Perception Backbone

To evaluate our perception-based policy, we now consider its navigation performance in simulation after training for 114 DAGger iterations (20000 labeled images generated), as well as when deployed on hardware.

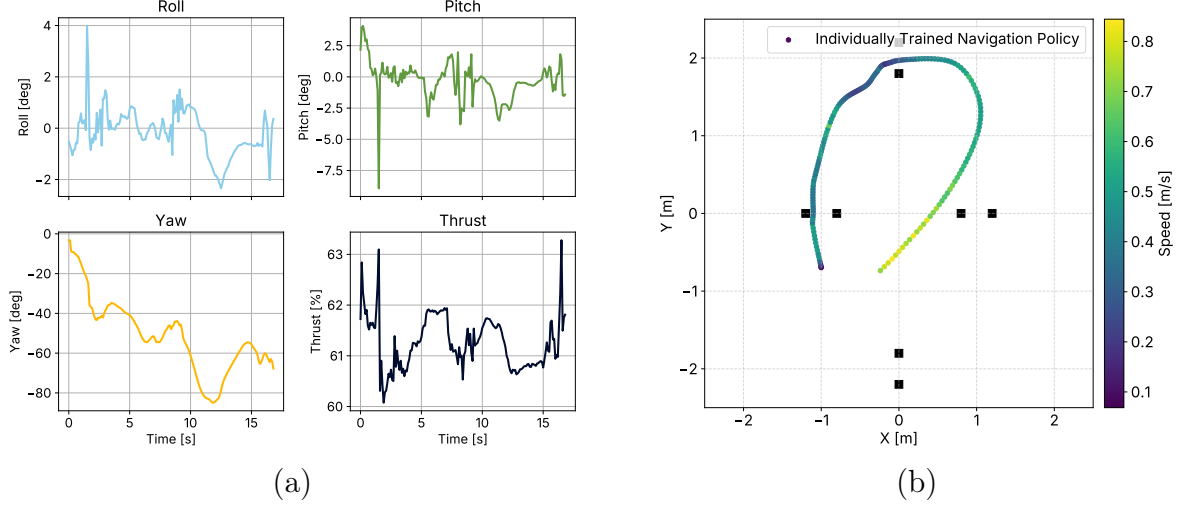


Figure 5.8: Navigation performance of the perception-fed policy in simulation. (a) Time series of the policy’s predicted roll, pitch, yaw, and thrust commands. (b) Resulting trajectory in the x-y plane, from start position at $\mathbf{p}_{\text{start}} = [-1, -0.8, 1]$ to terminating in a collision at $\mathbf{p}_{\text{coll}} = [-0.3, -0.8, 0]$. Racing gate locations are shown as black squares, and the color along the path encodes instantaneous velocity at each state.

While figure 5.8 (a) depicts the roll, pitch, yaw, and thrust axes’ time histories, figure 5.8 represents the 2D projection of the path taken on our simulated race track. Near the start state (0-6s), the policy behaves similarly to the experts’ navigation strategy depicted in 5.5. While thrusting and pitching at expert performance, roll and yaw show corrective behavior while passing the first gate. Around $t = 10\text{s}$ shortly before passing the second gate, the policy exhibits another correction to align with the gates’ normal and enable collision-free passing. Similar to what we observed during human-piloted experiments, this corrective behavior leads to a deceleration and overall slower navigation of the first two gates, when compared to the expert.

After passing the second gate, the perception-based policy does not manage to orient its yaw axis in order to face the third gate. Analysis of the yaw trajectories in Figure 5.8(a) shows that, unlike the expert policy, the student policy never exceeds a -90° rotation from its initial heading and even rebounds to an angle of -60° after passing the second gate. This backward-facing orientation prevents the drone from acquiring visual information on the third gate.

The failure arises from the planning networks’ dependence on accurate gate poses to generate valid actions. After the second gate, the camera’s field of view contains only the white walls of the flight space, as seen in figure 5.9. Image frames not containing racing gates were not represented in the perception policy’s training set, which entails that in the absence of gate features, the network produces incorrect detections, hallucinating gate poses that do not exist. This issue in vision-based drone racing without state estimation

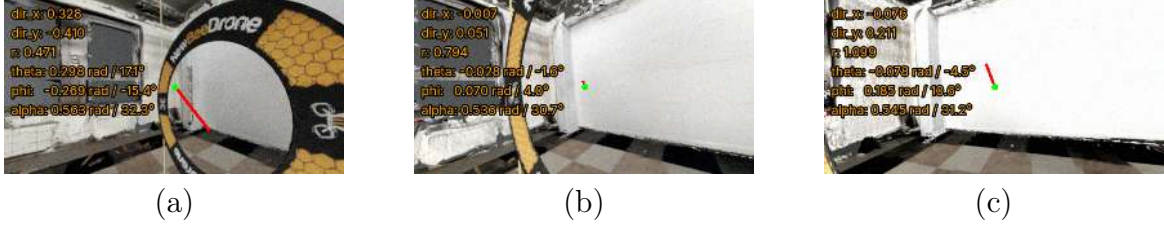


Figure 5.9: Three simulated images taken at different times upon passing the race tracks' second gate during a policy rollout. The perception policies gate pose prediction is printed on the left side of each image. The sequence of images illustrates the perceptual error after passing gate two, which leads to defective navigation performance. (a) Image taken at $t_{\text{step}} = 50$ as the first out of the displayed three. While a racing gate is visible on the right side of the image, the perception policy manages to predict its pose, indicated by the gate direction vector represented as a red line, as well as the remaining predicted values printed to the left side of the image. (b) Consecutive image taken at $t_{\text{step}} = 55$ with the racing gate not fully visible anymore but being clipped due to the camera's limited focal length. (c) Final image of the gate passage, taken at $t_{\text{step}} = 58$, represents the exit of the gate passage. While no image is visible during this frame, the perception policy hallucinates a gate pose in front of the camera frame.

has been previously documented [75] and highlights the importance of track designs that ensure valid gate perception throughout the course.

Furthermore, this perceptual challenge is the reason that our end-to-end trained policy would successfully navigate the race track since it overfitted to image features not related to the racing gates. However, this overfitting would limit its transferability.

Although the trained perception-based policy would not fully navigate the race track in simulation, we carried out hardware tests to validate transferability for the case of passing the first two gates of the race track.

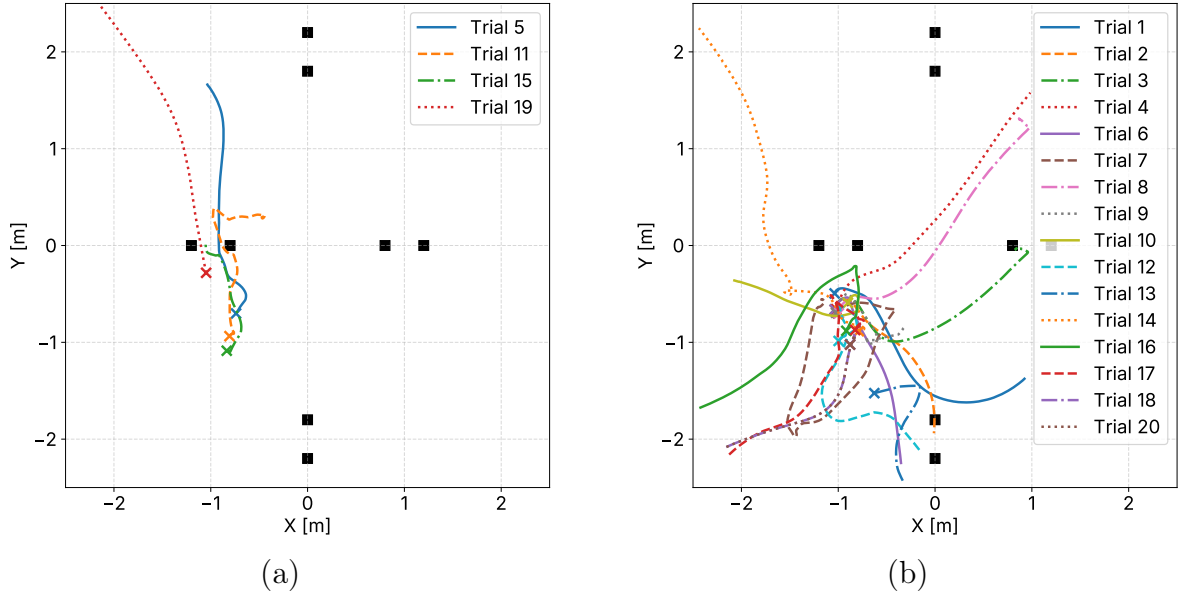


Figure 5.10: Flight performance of the perception-based policy over 20 test runs, with each trajectory distinguished by line style and trial index. (a) Four trajectories projected onto the x-y plane, launched from perturbed start positions around $[-1, 0.2, 1]$. All four runs successfully traverse the first racing gate but then diverge off course and crash before reaching the second gate. (b) Horizontal projections in the x-y plane of the remaining 16 flights, none of which clear the first gate before crashing.

Figure 5.10 depicts the perception-based policy’s performance on navigating the race track. Out of 20 conducted test flights, none would pass both gates one and two. Merely four trials successfully navigated through the first gate, but failed to plan towards the second, as seen in figure 5.10 (a). During the remaining 16 trials, the quadrotor would exhibit uncoordinated behavior. While at times locking onto other racing gates on the track, as seen in Trial 3 in Figure 5.10 (b), all executed tests ended in crashes without an evident trajectory pattern.

During the 20 experiments, we perturbed the starting position of the Crazyflie in x and y in order to figure out a configuration with a high success rate. From the results in Figure 5.10 (a), we could not identify a correlation between starting position and successful gate passage.

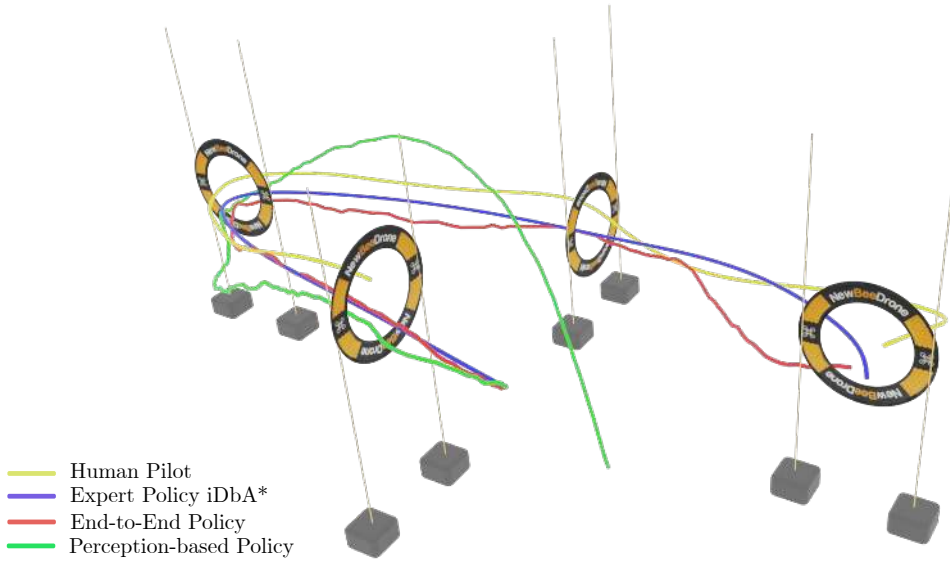


Figure 5.11: Rendering of the proposed race track. Each flight’s trajectory that was recorded during our experiments is indicated by a line with a distinct color. Visualized are a human-piloted lap (completed in median lap time), iDbA*’s computed expert trajectory, the end-to-end policy’s path in simulation, and the perception-based policy’s simulated trajectory.

5.3.4 Quantitative Evaluation

Following the qualitative evaluation of each experiment in sections 5.3.1, 5.3.2 and 5.3.3, we will now provide a quantitative evaluation regarding our proposed metrics: traveled distance d_{lap} , mean lap time $\bar{T}$, minimum lap time T_{min} , crash rate C , crash-per-gate C_g rate and the sim-to-real transfer gaps $\Delta C, \Delta C_g$.

Traveled Distance

To evaluate racing efficiency, we compare the total path lengths traversed by the three agents that successfully completed the course: human pilot, iDbA* expert planner in simulation, and our end-to-end policy in simulation.

The human pilot’s trajectory takes the longest path at 10.33m, due to wide cornering and corrective realignment maneuvers, as seen in figure 5.12. In contrast, the end-to-end policy

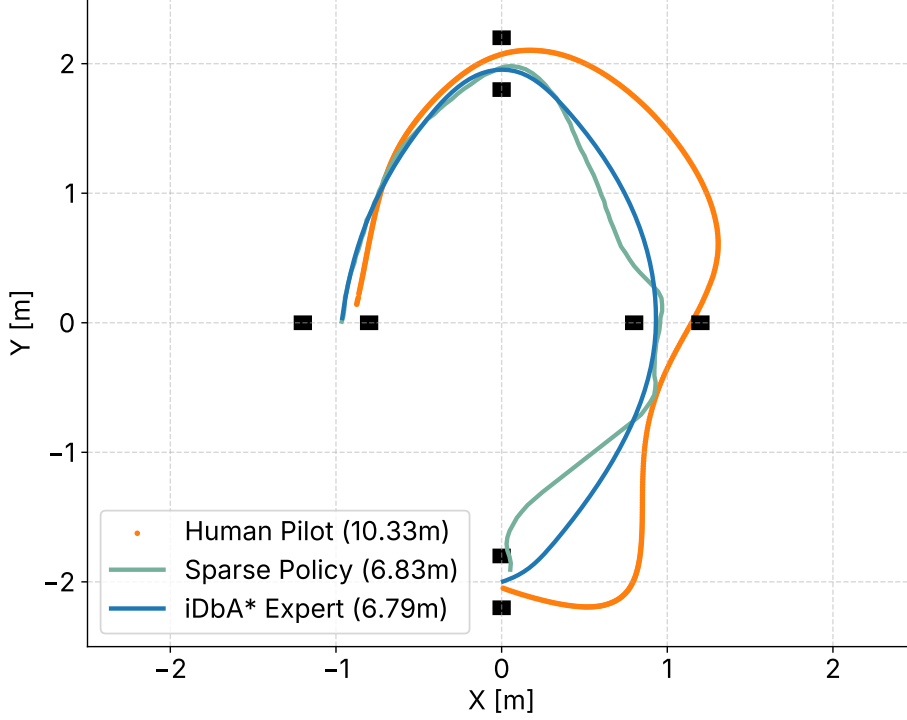


Figure 5.12: 2D projection of the different experiments’ recorded trajectories in the x-y plane. Depicted are a selected human-piloted flight with average lap time, the computed path by iDbA*, as well as the predicted trajectory by our end-to-end trained policy in simulation. Each trajectory can be identified by color and has its corresponding traveled distance throughout the race track annotated in the legend.

follows a notably shorter path of 6.83m, reflecting tighter turns and minimal correction. However, this policy executes a rapid, straight approach upon reaching the goal region, which is likely complicating the successful passage of the final gate. The iDbA* planner produces the shortest trajectory, measuring 6.79m, which is consistent with its optimality guarantees and ensures feasible gate passage on the last gate.

The minimal difference between the imitation policy’s path length to that of the expert planner demonstrates that our DAGger-trained policy achieves expert-level efficiency and significantly outperforms the human pilot in terms of total distance.

Lap Completion Time

To evaluate lap-time performance, we restrict our analysis to the three policies capable of completing the track without crashing: the simulated expert policy iDbA*, the human pilot, and the simulated end-to-end trained policy. For the simulated policies, mean lap time and minimum lap time coincide.

Lap times T are measured as the elapsed time between takeoff at $\mathbf{p}_{\text{start}} = [-1, -0.8, 1]$, immediately before gate one, and crossing the entrance of gate three at $\mathbf{p}_{\text{goal}} = [0, -2, 1]$. As represented in figure 5.13, the mean lap time $\bar{T}$ over 20 human-piloted laps was 20.76s, with a best lap of

$$T_{\min} = 18.64 \text{ s.}$$

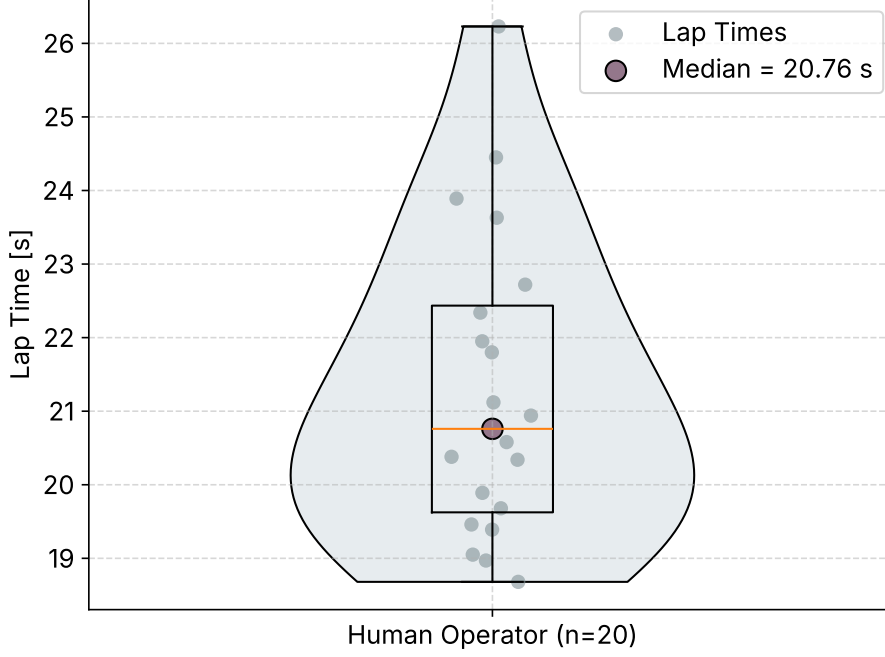


Figure 5.13: Violin plot depicting the human pilot’s lap times while navigating the race track. The shaded violin shows the full kernel density estimate of lap times with an overlaid boxplot summarizing the median, interquartile range, and whiskers.

The iDbA* expert policy achieved

$$T_{\min} = 17.51 \text{ s},$$

while our end-to-end policy was fastest, with

$$T_{\min} = 14.89 \text{ s}.$$

However, as illustrated in Figure 5.6 (b), the learned end-to-end policy exhibits high-velocity maneuvers during the second half of the course, which could not be achieved for the expert due to its limited motion primitives set. Consequently, a racing efficiency comparison is meaningful only between the iDbA* and the human pilot. In this analysis, the expert policy and the imitated policy outperform the human in both average lap time $\bar{T}$ and best lap time $T_{\min}$, reflecting their ability to generate trajectories with better racing performance.

Crash Rates and Sim-to-Real Transferability

The end-to-end policy exhibited robustness in simulation, completing a single trial without any collisions ($C_{\text{sim}} = 0$) and successfully traversing all four gates ($C_{g,\text{sim}} = 0$). However, when deployed on the hardware, it failed in every one of the 20 trials ($C_{\text{real}} = 1$), managing to pass only the first gate in 4 out of 20 attempts, which corresponds to a per-gate crash rate of

$$C_{g,\text{real}} = \frac{20}{4} = 5$$

Consequently, the sim-to-real transfer gaps amount to $\Delta C = 1$ and

$$\Delta C_g = 5 - 0 = 5$$

The loss of robustness on the hardware is illustrated by the hardware flight trajectories in Figure 5.6.

The perception-based policy also underperformed relative to simulation. In simulation it crashed once in one trial ($C_{\text{sim}} = 1$) while passing two gates ($C_{g,\text{sim}} = 1/2 = 0.5$). On hardware, the policy crashed in all 20 trials ($C_{\text{real}} = 1$), and successfully traversed only the first gate in 4 trials, yielding

$$C_{g,\text{real}} = \frac{20}{4} = 5$$

Thus, its transfer gaps are $\Delta C = 0$ and

$$\Delta C_g = 5 - 0.5 = 4.5$$

These results indicate that, despite the implementation of explicit gate detection, the perception-based policy suffers from similarly severe sim-to-real degradation, as shown by the trajectories in Figure 5.10.

In both cases, the increase in per-gate crash rates upon hardware deployment highlights a substantial sim-to-real transfer gap.

Conclusion of Quantitative Evaluation

The quantitative analysis confirms that the iDbA* motion planner achieves the highest efficiency in simulation, exhibiting the shortest lap distance d_{lap} and the lowest mean and minimum lap times $\bar{T}$, T_{min} (excluding the end-to-end policy, due to infeasible velocities). This observation is consistent with its optimality guarantees. The simulated DAgger-trained end-to-end policy ranks second, with both path length and completion time very close to the expert planner. The human pilot demonstrates substantially greater lap distance and slower mean lap time, although its single fastest lap approaches expert performance.

While the metrics d_{lap} , $\bar{T}$, and T_{min} provide clear differentiation of agent performance, the crash-rate metrics C and C_g are less appropriate for a direct comparison between simulation and hardware. In simulation, trials produce deterministic outcomes with zero crashes, whereas real-world flights are subject to disturbances not present in simulation. Furthermore, because none of the autonomous policies completed a single lap on the race track, the sim-to-real crash gaps ΔC and ΔC_g are high, which attenuates the significance of a quantitative classification of the sim-to-real gap. Nevertheless, these high values quantify the severe performance degradation when transferring from simulation to physical flight.

6. Discussion and Conclusion

In this chapter, we discuss the experimental findings and critically examine the main limitations encountered as well as interpret the implications of our results. Based on these insights, we then outline future actions for improving our vision-to-action drone racing policy and finally conclude this work.

6.1 Implications and Limitations

The experimental results demonstrate that the iDbA* expert planning algorithm outperforms a human pilot in both racing efficiency and speed on the proposed track. Moreover, our end-to-end vision-to-action policy, trained via DAgger, achieves performance comparable to the iDbA* planner and exceeds the human reference in terms of path length and lap time when simulated.

Despite its success in simulation, the end-to-end policy fails to transfer to hardware, showing extreme performance degradation. We attribute this failure to overfitting on simulation-specific features in combination with the sim-to-real discrepancy.

To address this limitation, we developed a perception-based policy that relies on detected gate poses rather than raw images. However, this policy is unable to complete the course in simulation and on hardware, due to racing gates not being visible in the camera’s field of view after the first turn, resulting in false predictions. This highlights a critical shortcoming in our approach, where the perception backbone was not exposed to negative examples, such as images without any visible gates.

An additional factor contributing to poor hardware performance may be the Bitcraze Crazyflie’s onboard PID controller. Tracking errors related to the controllers’ performance may cause deviations from the planned trajectories. Furthermore, the iDbA* planner itself employs a double-integrator approximation of quadrotor dynamics, as shown in section 2.2.3. This simplification may limit its ability to generate truly feasible trajectories under real-world conditions.

Aside from our intended study, we observed remarkable durability of the Crazyflie platform. Over fifty crashes did not compromise the quadrotor’s flight capabilities, underscoring the robustness of the hardware for aggressive autonomous operations.

6.2 Future Work

To frame our future work, we first outline essential additions to our approach and then present short-term and long-term plans to improve our policy’s racing performance.

In the absence of prior track geometry, the system must both explore and localize gates. To address this, a search strategy can be implemented that activates when the next gate is lost, for instance, commanding the drone to rotate in place around its yaw axis until the gate re-enters the field of view.

Additionally, one may augment the perception policy by providing the onboard EKF’s state estimate. This functionality is already provided by the BitCraze firmware and thus could be integrated quickly for the Crazyflie quadrotor. The additional input to the planning policy alongside raw image and body-velocities is likely to reduce the sim-to-real gap, and enable more precise gate-to-gate navigation as demonstrated in [11]. With global position information, trajectory learning is simplified and may improve overall flight aggressiveness without relying solely on image cues.

Short-term efforts should focus on mitigating overfitting to the simulation environment. By collecting real-world image data and labeling each frame with ground-truth state information using a motion capture system, one can augment the dataset that the policy is trained on. Furthermore, extending the policy’s observation space to include on-board EKF state estimates and replacing the existing PID–double-integrator controller with the full inverse-dynamics law are implemented quickly and are likely to improve trajectory learning and tracking precision.

Longer-term development may include providing the full quadrotor dynamics model to the iDbA* motion planner, as well as for state propagation in simulation to mitigate the sim-to-real gap by avoiding reliance on the dynamics’ approximation. As a concluding effort, integrating a full mapping submodule similar to the ones employed by the autonomous drone racing teams in the AlphaPilot Challenge [11], [48], for instance, by implementing a global mapping strategy such as occupancy-grid or pose-graph SLAM, provides the policy with global state information that is likely to drastically improve racing performance, as well as sim-to-real transfer.

6.3 Conclusion

In this work, we developed and evaluated two autonomous vision-to-action drone racing policies trained via the imitation learning paradigm DAgger. We trained each policy by imitating the expert algorithm iDbA*, which offers locally optimal motion planning.

The first implemented policy was trained end-to-end feeding in raw images to predict actions, which in simulation outperformed a human pilot in both lap time and path length. The second developed policy employs a learned perception module to infer gate poses from onboard images, feeding these estimates into the separate planning policy. While both approaches demonstrated good performance in simulation, neither transferred to the physical platform. The end-to-end approach suffered from overfitting to synthetic visual features, whereas the perception-based neural network failed when gates exited the camera’s view.

We identified two critical challenges for sim-to-real transfer in high-speed autonomy. First, overfitting, where the end-to-end network would learn to predict actions from background image features unique to the simulation environment. This was mitigated by employing the perception-based policy, which extracts only the relevant gate features.

Second, maintaining robust state awareness when gates exit the camera image. Future efforts should focus on integrating a mapping subsystem to deliver the vehicle’s state estimates and out-of-frame gate poses to a planning policy while retaining our perception backbone to detect the gates when visible.

Overall, this thesis contributes insights into the trade-offs between end-to-end and perception-driven autonomy pipelines for aggressive quadrotor navigation and provides findings on the limitations of the transferability of autonomous drone racing policies.

Bibliography

- [1] E. Kaufmann, L. Bauersfeld, A. Loquercio, M. Müller, V. Koltun, and D. Scaramuzza, “Champion-level drone racing using deep reinforcement learning,” en, *Nature*, vol. 620, no. 7976, pp. 982–987, Aug. 2023, Publisher: Nature Publishing Group, ISSN: 1476-4687. DOI: 10.1038/s41586-023-06419-4. [Online]. Available: <https://www.nature.com/articles/s41586-023-06419-4> (visited on 04/02/2025).
- [2] A. S. Huang, A. Bachrach, P. Henry, *et al.*, “Visual Odometry and Mapping for Autonomous Flight Using an RGB-D Camera,” en, Jan. 2011. [Online]. Available: <https://openreview.net/forum?id=0i40wvtXQa> (visited on 06/05/2025).
- [3] K. S. Lee, M. Ovinis, T. Nagarajan, R. Seulin, and O. Morel, “Autonomous patrol and surveillance system using unmanned aerial vehicles,” in *2015 IEEE 15th International Conference on Environment and Electrical Engineering (EEEIC)*, Jun. 2015, pp. 1291–1297. DOI: 10.1109/EEEIC.2015.7165356. [Online]. Available: <https://ieeexplore.ieee.org/document/7165356> (visited on 06/05/2025).
- [4] O. Araar and N. Aouf, “Visual servoing of a Quadrotor UAV for autonomous power lines inspection,” in *22nd Mediterranean Conference on Control and Automation*, Jun. 2014, pp. 1418–1424. DOI: 10.1109/MED.2014.6961575. [Online]. Available: <https://ieeexplore.ieee.org/document/6961575> (visited on 06/05/2025).
- [5] D. K. D. Villa, A. S. Brandão, and M. Sarcinelli-Filho, “A Survey on Load Transportation Using Multirotor UAVs,” en, *Journal of Intelligent & Robotic Systems*, vol. 98, no. 2, pp. 267–296, May 2020, ISSN: 1573-0409. DOI: 10.1007/s10846-019-01088-w. [Online]. Available: <https://doi.org/10.1007/s10846-019-01088-w> (visited on 06/05/2025).
- [6] A. Krizhevsky, I. Sutskever, and G. E. Hinton, “ImageNet Classification with Deep Convolutional Neural Networks,” in *Advances in Neural Information Processing Systems*, F. Pereira, C. J. Burges, L. Bottou, and K. Q. Weinberger, Eds., vol. 25, Curran Associates, Inc., 2012. [Online]. Available: https://proceedings.neurips.cc/paper_files/paper/2012/file/c399862d3b9d6b76c8436e924a68c45b-Paper.pdf.
- [7] S. Ross, G. J. Gordon, and J. A. Bagnell, *A Reduction of Imitation Learning and Structured Prediction to No-Regret Online Learning*, arXiv:1011.0686 [cs], Mar. 2011. DOI: 10.48550/arXiv.1011.0686. [Online]. Available: <http://arxiv.org/abs/1011.0686> (visited on 04/20/2025).
- [8] V. Mnih, K. Kavukcuoglu, D. Silver, *et al.*, *Playing Atari with Deep Reinforcement Learning*, arXiv:1312.5602 [cs], Dec. 2013. DOI: 10.48550/arXiv.1312.5602. [Online]. Available: <http://arxiv.org/abs/1312.5602> (visited on 06/06/2025).

-
- [9] M. Campbell, A. J. Hoane, and F.-h. Hsu, “Deep Blue,” *Artificial Intelligence*, vol. 134, no. 1, pp. 57–83, Jan. 2002, ISSN: 0004-3702. DOI: 10.1016/S0004-3702(01)00129-1. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S0004370201001291> (visited on 06/05/2025).
 - [10] D. Silver, A. Huang, C. J. Maddison, *et al.*, “Mastering the game of Go with deep neural networks and tree search,” en, *Nature*, vol. 529, no. 7587, pp. 484–489, Jan. 2016, Publisher: Nature Publishing Group, ISSN: 1476-4687. DOI: 10.1038/nature16961. [Online]. Available: <https://www.nature.com/articles/nature16961> (visited on 06/05/2025).
 - [11] C. D. Wagter, F. Paredes-Vallés, N. Sheth, and G. d. Croon, “The Artificial Intelligence behind the winning entry to the 2019 AI Robotic Racing Competition,” *Field Robotics*, vol. 2, pp. 1263–1290, Jun. 2022, arXiv:2109.14985 [cs], ISSN: 2771-3989. DOI: 10.55417/fr.2022042. [Online]. Available: <http://arxiv.org/abs/2109.14985> (visited on 05/13/2025).
 - [12] F. D and W. Rj, “Science, technology and the future of small autonomous drones,” en, *Nature*, vol. 521, no. 7553, May 2015, Publisher: Nature, ISSN: 1476-4687. DOI: 10.1038/nature14542. [Online]. Available: <https://pubmed.ncbi.nlm.nih.gov/26017445/> (visited on 06/05/2025).
 - [13] J. Betz, H. Zheng, A. Liniger, *et al.*, “Autonomous Vehicles on the Edge: A Survey on Autonomous Vehicle Racing,” *IEEE Open Journal of Intelligent Transportation Systems*, vol. 3, pp. 458–488, 2022, arXiv:2202.07008 [cs], ISSN: 2687-7813. DOI: 10.1109/ojits.2022.3181510. [Online]. Available: <http://arxiv.org/abs/2202.07008> (visited on 06/05/2025).
 - [14] D. Hanover, A. Loquercio, L. Bauersfeld, *et al.*, “Autonomous Drone Racing: A Survey,” en, *IEEE Transactions on Robotics*, vol. 40, pp. 3044–3067, 2024, arXiv:2301.01755 [cs], ISSN: 1552-3098, 1941-0468. DOI: 10.1109/TR0.2024.3400838. [Online]. Available: <http://arxiv.org/abs/2301.01755> (visited on 04/11/2025).
 - [15] T. A. H. Wilkinson, *Early Dynastic Egypt*. London: Routledge, Sep. 2002, ISBN: 978-0-203-02438-6. DOI: 10.4324/9780203024386.
 - [16] Z. Ameli, Y. Aremanda, W. A. Friess, and E. N. Landis, “Impact of UAV Hardware Options on Bridge Inspection Mission Capabilities,” en, *Drones*, vol. 6, no. 3, p. 64, Mar. 2022, Number: 3 Publisher: Multidisciplinary Digital Publishing Institute, ISSN: 2504-446X. DOI: 10.3390/drones6030064. [Online]. Available: <https://www.mdpi.com/2504-446X/6/3/64> (visited on 06/06/2025).
 - [17] J. Ortiz-Haro, W. Hoenig, V. N. Hartmann, and M. Toussaint, *iDb-A*: Iterative Search and Optimization for Optimal Kinodynamic Motion Planning*, arXiv:2311.03553 [cs], Nov. 2023. DOI: 10.48550/arXiv.2311.03553. [Online]. Available: <http://arxiv.org/abs/2311.03553> (visited on 04/20/2025).
 - [18] S. Li, E. v. d. Horst, P. Duernay, C. D. Wagter, and G. C. H. E. d. Croon, “Visual Model-predictive Localization for Computationally Efficient Autonomous Racing of a 72-gram Drone,” *Journal of Field Robotics*, vol. 37, no. 4, pp. 667–692, Jun. 2020, arXiv:1905.10110 [cs], ISSN: 1556-4959, 1556-4967. DOI: 10.1002/rob.21956. [Online]. Available: <http://arxiv.org/abs/1905.10110> (visited on 07/03/2025).
 - [19] D. Mellinger and V. Kumar, “Minimum snap trajectory generation and control for quadrotors,” in *2011 IEEE International Conference on Robotics and Automation*, ISSN: 1050-4729, May 2011, pp. 2520–2525. DOI: 10.1109/ICRA.2011.5980409.
-

- [Online]. Available: <https://ieeexplore.ieee.org/document/5980409/> (visited on 04/08/2025).
- [20] M. v. Nieuwstadt and R. M. Murray, "Real Time Trajectory Generation for Differentially Flat Systems," *IFAC Proceedings Volumes*, 13th World Congress of IFAC, 1996, San Francisco USA, 30 June - 5 July, vol. 29, no. 1, pp. 2301–2306, Jun. 1996, ISSN: 1474-6670. DOI: 10.1016/S1474-6670(17)58016-7. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S1474667017580167> (visited on 04/25/2025).
- [21] M. Faessler, A. Franchi, and D. Scaramuzza, "Differential Flatness of Quadrotor Dynamics Subject to Rotor Drag for Accurate Tracking of High-Speed Trajectories," *IEEE Robotics and Automation Letters*, vol. 3, no. 2, pp. 620–626, Apr. 2018, arXiv:1712.02402 [cs], ISSN: 2377-3766, 2377-3774. DOI: 10.1109/LRA.2017.2776353. [Online]. Available: <http://arxiv.org/abs/1712.02402> (visited on 04/08/2025).
- [22] R. Amin, L. Aijun, and S. Shamshirband, "A review of quadrotor UAV: Control methodologies and performance evaluation," *International Journal of Automation and Control*, vol. 10, no. 2, pp. 87–103, Jan. 2016, Publisher: Inderscience Publishers, ISSN: 1740-7516. DOI: 10.1504/IJAAC.2016.076453. [Online]. Available: <https://www.inderscienceonline.com/doi/abs/10.1504/IJAAC.2016.076453> (visited on 04/08/2025).
- [23] A. Khalid, Z. Mushtaq, S. Arif, K. Zeb, M. A. Khan, and S. Bakshi, "Control Schemes for Quadrotor UAV: Taxonomy and Survey," *ACM Comput. Surv.*, vol. 56, no. 5, 124:1–124:32, Nov. 2023, ISSN: 0360-0300. DOI: 10.1145/3617652. [Online]. Available: <https://doi.org/10.1145/3617652> (visited on 04/08/2025).
- [24] N. Michael, D. Mellinger, Q. Lindsey, and V. Kumar, "The GRASP Multiple Micro-UAV Testbed," *IEEE Robotics & Automation Magazine*, vol. 17, no. 3, pp. 56–65, Sep. 2010, ISSN: 1558-223X. DOI: 10.1109/MRA.2010.937855. [Online]. Available: <https://ieeexplore.ieee.org/document/5569026> (visited on 04/25/2025).
- [25] Bitcraze, *Crazyflie firmware: Controllers*, Accessed: 2025-06-14, 2024. [Online]. Available: <https://www.bitcraze.io/documentation/repository/crazyflie-firmware/master/functional-areas/sensor-to-control/controllers/>.
- [26] S. Bouabdallah, P. Murrieri, and R. Siegwart, "Design and control of an indoor micro quadrotor," in *IEEE International Conference on Robotics and Automation, 2004. Proceedings. ICRA '04. 2004*, ISSN: 1050-4729, vol. 5, Apr. 2004, 4393–4398 Vol.5. DOI: 10.1109/ROBOT.2004.1302409. [Online]. Available: <https://ieeexplore.ieee.org/document/1302409> (visited on 06/24/2025).
- [27] H. Bolandi, M. Rezaei, R. Mohsenipour, H. Nemati, and S. M. Smailzadeh, "Attitude Control of a Quadrotor with Optimized PID Controller," in *Intelligent Control and Automation*, vol. 4, no. 3, pp. 335–342, Aug. 2013, Number: 3 Publisher: Scientific Research Publishing. DOI: 10.4236/ica.2013.43039. [Online]. Available: <https://www.scirp.org/journal/paperinformation?paperid=35654> (visited on 06/24/2025).
- [28] M. Bangura and R. Mahony, "Real-time Model Predictive Control for Quadrotors," *IFAC Proceedings Volumes*, 19th IFAC World Congress, vol. 47, no. 3, pp. 11773–11780, Jan. 2014, ISSN: 1474-6670. DOI: 10.3182/20140824-6-ZA-1003.00203. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S1474667016434890> (visited on 06/24/2025).

-
- [29] S. Sun, A. Romero, P. Foehn, E. Kaufmann, and D. Scaramuzza, *A Comparative Study of Nonlinear MPC and Differential-Flatness-Based Control for Quadrotor Agile Flight*, arXiv:2109.01365 [cs], Jan. 2024. DOI: 10.48550/arXiv.2109.01365. [Online]. Available: <http://arxiv.org/abs/2109.01365> (visited on 06/24/2025).
 - [30] P. Kunapuli, J. Welde, D. Jayaraman, and V. Kumar, *Leveling the Playing Field: Carefully Comparing Classical and Learned Controllers for Quadrotor Trajectory Tracking*, arXiv:2506.17832 [cs], Jun. 2025. DOI: 10.48550/arXiv.2506.17832. [Online]. Available: <http://arxiv.org/abs/2506.17832> (visited on 07/03/2025).
 - [31] S. M. LaValle and J. J. Kuffner Jr., “Randomized Kinodynamic Planning,” EN, *The International Journal of Robotics Research*, vol. 20, no. 5, pp. 378–400, May 2001, Publisher: SAGE Publications Ltd STM, ISSN: 0278-3649. DOI: 10.1177/02783640122067453. [Online]. Available: <https://doi.org/10.1177/02783640122067453> (visited on 05/02/2025).
 - [32] Y. Li, Z. Littlefield, and K. E. Bekris, “Asymptotically optimal sampling-based kinodynamic planning,” EN, *The International Journal of Robotics Research*, vol. 35, no. 5, pp. 528–564, Apr. 2016, Publisher: SAGE Publications Ltd STM, ISSN: 0278-3649. DOI: 10.1177/0278364915614386. [Online]. Available: <https://doi.org/10.1177/0278364915614386> (visited on 05/02/2025).
 - [33] Z. Littlefield and K. E. Bekris, “Efficient and Asymptotically Optimal Kinodynamic Motion Planning via Dominance-Informed Regions,” in *2018 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, ISSN: 2153-0866, Oct. 2018, pp. 1–9. DOI: 10.1109/IROS.2018.8593672. [Online]. Available: <https://ieeexplore.ieee.org/document/8593672/> (visited on 05/02/2025).
 - [34] M. Pivtoraiko and A. Kelly, “Kinodynamic motion planning with state lattice motion primitives,” in *2011 IEEE/RSJ International Conference on Intelligent Robots and Systems*, ISSN: 2153-0866, Sep. 2011, pp. 2172–2179. DOI: 10.1109/IROS.2011.6094900. [Online]. Available: <https://ieeexplore.ieee.org/document/6094900/> (visited on 05/02/2025).
 - [35] M. Likhachev, D. Ferguson, and F. Ave, “Planning Long Dynamically-Feasible Maneuvers for Autonomous Vehicles,” en,
 - [36] J. Schulman, Y. Duan, J. Ho, *et al.*, “Motion planning with sequential convex optimization and convex collision checking,” en, *The International Journal of Robotics Research*, vol. 33, no. 9, pp. 1251–1270, Aug. 2014, ISSN: 0278-3649, 1741-3176. DOI: 10.1177/0278364914528132. [Online]. Available: <https://journals.sagepub.com/doi/10.1177/0278364914528132> (visited on 05/02/2025).
 - [37] R. Bonalli, A. Cauligi, A. Bylard, and M. Pavone, “GuSTO: Guaranteed Sequential Trajectory optimization via Sequential Convex Programming,” in *2019 International Conference on Robotics and Automation (ICRA)*, ISSN: 2577-087X, May 2019, pp. 6741–6747. DOI: 10.1109/ICRA.2019.8794205. [Online]. Available: <https://ieeexplore.ieee.org/document/8794205/> (visited on 05/02/2025).
 - [38] M. Toussaint, “A Tutorial on Newton Methods for Constrained Trajectory Optimization and Relations to SLAM, Gaussian Process Smoothing, Optimal Control, and Probabilistic Inference,” en, in *Geometric and Numerical Foundations of Movements*, J.-P. Laumond, N. Mansard, and J.-B. Lasserre, Eds., vol. 117, Series Title: Springer Tracts in Advanced Robotics, Cham: Springer International Publishing, 2017, pp. 361–392, ISBN: 978-3-319-51546-5 978-3-319-51547-2. DOI: 10.1007/978-3-319-51547-2_15. [Online]. Available: http://link.springer.com/10.1007/978-3-319-51547-2_15 (visited on 05/02/2025).
-

-
- [39] W. Hoenig, J. Ortiz-Haro, and M. Toussaint, *Db-A*: Discontinuity-bounded Search for Kinodynamic Mobile Robot Motion Planning*, arXiv:2203.11108 [cs], Aug. 2022. DOI: 10.48550/arXiv.2203.11108. [Online]. Available: <http://arxiv.org/abs/2203.11108> (visited on 05/11/2025).
 - [40] I. Noreen, A. Khan, and Z. Habib, “Optimal path planning using rrt* based approaches: A survey and future directions,” *International Journal of Advanced Computer Science and Applications*, vol. 7, no. 11, 2016.
 - [41] J. Meng, V. M. Pawar, S. Kay, and A. Li, “UAV Path Planning System Based on 3D Informed RRT* for Dynamic Obstacle Avoidance,” in *2018 IEEE International Conference on Robotics and Biomimetics (ROBIO)*, Dec. 2018, pp. 1653–1658. DOI: 10.1109/ROBIO.2018.8665162. [Online]. Available: <https://ieeexplore.ieee.org/document/8665162> (visited on 07/03/2025).
 - [42] S. LaValle, “Rapidly-exploring random trees : A new tool for path planning,” *The annual research report*, 1998. [Online]. Available: <https://www.semanticscholar.org/paper/Rapidly-exploring-random-trees-%3A-a-new-tool-for-LaValle/d967d9550f831a8b3f5cb00f8835a4c866da60ad> (visited on 06/25/2025).
 - [43] A. Boeuf, J. Cortés, R. Alami, and T. Siméon, “Planning agile motions for quadrotors in constrained environments,” in *2014 IEEE/RSJ International Conference on Intelligent Robots and Systems*, ISSN: 2153-0866, Sep. 2014, pp. 218–223. DOI: 10.1109/IRoS.2014.6942564. [Online]. Available: <https://ieeexplore.ieee.org/document/6942564> (visited on 06/25/2025).
 - [44] S. Schaal, “Is imitation learning the route to humanoid robots?” *Trends in Cognitive Sciences*, vol. 3, no. 6, pp. 233–242, Jun. 1999, ISSN: 1364-6613. DOI: 10.1016/S1364-6613(99)01327-3. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S1364661399013273> (visited on 04/27/2025).
 - [45] J. Ho and S. Ermon, *Generative Adversarial Imitation Learning*, arXiv:1606.03476 [cs], Jun. 2016. DOI: 10.48550/arXiv.1606.03476. [Online]. Available: <http://arxiv.org/abs/1606.03476> (visited on 06/25/2025).
 - [46] S. Bandela and Y. Cao, “Drone Navigation in Unreal Engine Using Generative Adversarial Imitation Learning,” in *AIAA SCITECH 2023 Forum*, eprint: <https://arc.aiaa.org/doi/pdf/10.2514/6.2023-0506>, American Institute of Aeronautics and Astronautics, 2023. DOI: 10.2514/6.2023-0506. [Online]. Available: <https://arc.aiaa.org/doi/abs/10.2514/6.2023-0506> (visited on 07/03/2025).
 - [47] P. Foehn, D. Brescianini, E. Kaufmann, *et al.*, *AlphaPilot: Autonomous Drone Racing*, arXiv:2005.12813 [cs], Aug. 2021. DOI: 10.48550/arXiv.2005.12813. [Online]. Available: <http://arxiv.org/abs/2005.12813> (visited on 05/13/2025).
 - [48] K. Lee, J. Gibson, and E. A. Theodorou, *Aggressive Perception-Aware Navigation using Deep Optical Flow Dynamics and PixelMPC*, arXiv:2001.02307 [cs], Jan. 2020. DOI: 10.48550/arXiv.2001.02307. [Online]. Available: <http://arxiv.org/abs/2001.02307> (visited on 05/15/2025).
 - [49] S. Jung, S. Hwang, H. Shin, and D. H. Shim, “Perception, Guidance, and Navigation for Indoor Autonomous Drone Racing Using Deep Learning,” *IEEE Robotics and Automation Letters*, vol. 3, no. 3, pp. 2539–2544, Jul. 2018, ISSN: 2377-3766. DOI: 10.1109/LRA.2018.2808368. [Online]. Available: <https://ieeexplore.ieee.org/abstract/document/8299437> (visited on 05/15/2025).
 - [50] A. M. C. Rezende, V. R. F. Miranda, P. A. F. Rezeck, *et al.*, “An integrated solution for an autonomous drone racing in indoor environments,” in *Intelligent Service Robotics*, vol. 14, no. 5, pp. 641–661, Nov. 2021, ISSN: 1861-2784. DOI:
-

- 10.1007/s11370-021-00385-4. [Online]. Available: <https://doi.org/10.1007/s11370-021-00385-4> (visited on 05/15/2025).
- [51] D. Scaramuzza and Z. Zhang, *Visual-Inertial Odometry of Aerial Robots*, arXiv:1906.03289 [cs], Jun. 2019. DOI: 10.48550/arXiv.1906.03289. [Online]. Available: <http://arxiv.org/abs/1906.03289> (visited on 05/12/2025).
- [52] Y. Yang, P. Geneva, X. Zuo, and G. Huang, *Online Self-Calibration for Visual-Inertial Navigation Systems: Models, Analysis and Degeneracy*, arXiv:2201.09170 [cs], Jan. 2022. DOI: 10.48550/arXiv.2201.09170. [Online]. Available: <http://arxiv.org/abs/2201.09170> (visited on 05/12/2025).
- [53] J. Engel, V. Koltun, and D. Cremers, “Direct Sparse Odometry,” *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 40, no. 3, pp. 611–625, Mar. 2018, ISSN: 1939-3539. DOI: 10.1109/TPAMI.2017.2658577. [Online]. Available: <https://ieeexplore.ieee.org/document/7898369> (visited on 05/12/2025).
- [54] S. Leutenegger, S. Lynen, M. Bosse, R. Siegwart, and P. Furgale, “Keyframe-based visual-inertial odometry using nonlinear optimization,” en, *The International Journal of Robotics Research*, vol. 34, no. 3, pp. 314–334, 2015, Accepted: 2017-06-11T16:59:38Z Publisher: SAGE, ISSN: 0278-3649. DOI: 10.1177/0278364914554813. [Online]. Available: <https://www.research-collection.ethz.ch/handle/20.500.11850/100060> (visited on 05/12/2025).
- [55] A. I. Mourikis and S. I. Roumeliotis, “A Multi-State Constraint Kalman Filter for Vision-aided Inertial Navigation,” in *Proceedings 2007 IEEE International Conference on Robotics and Automation*, ISSN: 1050-4729, Apr. 2007, pp. 3565–3572. DOI: 10.1109/ROBOT.2007.364024. [Online]. Available: <https://ieeexplore.ieee.org/document/4209642> (visited on 05/12/2025).
- [56] E. Kaufmann, M. Gehrig, P. Foehn, *et al.*, *Beauty and the Beast: Optimal Methods Meet Learning for Drone Racing*, arXiv:1810.06224 [cs], Mar. 2019. DOI: 10.48550/arXiv.1810.06224. [Online]. Available: <http://arxiv.org/abs/1810.06224> (visited on 05/12/2025).
- [57] A. R. Vidal, H. Rebecq, T. Horstschaefer, and D. Scaramuzza, “Ultimate SLAM? Combining Events, Images, and IMU for Robust Visual SLAM in HDR and High Speed Scenarios,” *IEEE Robotics and Automation Letters*, vol. 3, no. 2, pp. 994–1001, Apr. 2018, arXiv:1709.06310 [cs], ISSN: 2377-3766, 2377-3774. DOI: 10.1109/LRA.2018.2793357. [Online]. Available: <http://arxiv.org/abs/1709.06310> (visited on 05/12/2025).
- [58] S. Sun, G. Cioffi, C. d. Visser, and D. Scaramuzza, “Autonomous Quadrotor Flight despite Rotor Failure with Onboard Vision Sensors: Frames vs. Events,” *IEEE Robotics and Automation Letters*, vol. 6, no. 2, pp. 580–587, Apr. 2021, arXiv:2102.13406 [cs], ISSN: 2377-3766, 2377-3774. DOI: 10.1109/LRA.2020.3048875. [Online]. Available: <http://arxiv.org/abs/2102.13406> (visited on 05/12/2025).
- [59] B. Nisar, P. Foehn, D. Falanga, and D. Scaramuzza, “VIMO: Simultaneous Visual Inertial Model-Based Odometry and Force Estimation,” *IEEE Robotics and Automation Letters*, vol. 4, no. 3, pp. 2785–2792, Jul. 2019, ISSN: 2377-3766. DOI: 10.1109/LRA.2019.2918689. [Online]. Available: <https://ieeexplore.ieee.org/document/8721075> (visited on 05/12/2025).
- [60] H. Nguyen, M. Kamel, K. Alexis, and R. Siegwart, *Model Predictive Control for Micro Aerial Vehicles: A Survey*, arXiv:2011.11104 [cs], Nov. 2020. DOI: 10.48550/

- arXiv:2011.11104. [Online]. Available: <http://arxiv.org/abs/2011.11104> (visited on 05/13/2025).
- [61] G. Torrente, E. Kaufmann, P. Föhn, and D. Scaramuzza, “Data-Driven MPC for Quadrotors,” *IEEE Robotics and Automation Letters*, vol. 6, no. 2, pp. 3769–3776, Apr. 2021, ISSN: 2377-3766. DOI: 10.1109/LRA.2021.3061307. [Online]. Available: <https://ieeexplore.ieee.org/document/9361343> (visited on 05/13/2025).
- [62] G. Williams, N. Wagener, B. Goldfain, *et al.*, “Information theoretic MPC for model-based reinforcement learning,” in *2017 IEEE International Conference on Robotics and Automation (ICRA)*, May 2017, pp. 1714–1721. DOI: 10.1109/ICRA.2017.7989202. [Online]. Available: <https://ieeexplore.ieee.org/document/7989202/> (visited on 05/18/2025).
- [63] G. Costante, C. Forster, J. Delmerico, P. Valigi, and D. Scaramuzza, *Perception-aware Path Planning*, arXiv:1605.04151 [cs], Feb. 2017. DOI: 10.48550/arXiv.1605.04151. [Online]. Available: <http://arxiv.org/abs/1605.04151> (visited on 05/13/2025).
- [64] D. Falanga, E. Mueggler, M. Faessler, and D. Scaramuzza, “Aggressive quadrotor flight through narrow gaps with onboard sensing and computing using active vision,” in *2017 IEEE International Conference on Robotics and Automation (ICRA)*, May 2017, pp. 5774–5781. DOI: 10.1109/ICRA.2017.7989679. [Online]. Available: <https://ieeexplore.ieee.org/document/7989679> (visited on 05/13/2025).
- [65] M. Greeff, T. D. Barfoot, and A. P. Schoellig, “A Perception-Aware Flatness-Based Model Predictive Controller for Fast Vision-Based Multirotor Flight,” en, *IFAC-PapersOnLine*, vol. 53, no. 2, pp. 9412–9419, 2020, ISSN: 24058963. DOI: 10.1016/j.ifacol.2020.12.2411. [Online]. Available: <https://linkinghub.elsevier.com/retrieve/pii/S2405896320330937> (visited on 05/13/2025).
- [66] A. Bhattacharya, M. Cannici, N. Rao, *et al.*, *Monocular Event-Based Vision for Obstacle Avoidance with a Quadrotor*, arXiv:2411.03303 [cs], Nov. 2024. DOI: 10.48550/arXiv.2411.03303. [Online]. Available: <http://arxiv.org/abs/2411.03303> (visited on 05/16/2025).
- [67] H. X. Pham, A. Sarabakha, M. Odnoshyvkina, and E. Kayacan, *PencilNet: Zero-Shot Sim-to-Real Transfer Learning for Robust Gate Perception in Autonomous Drone Racing*, arXiv:2207.14131 [cs], Jul. 2022. DOI: 10.48550/arXiv.2207.14131. [Online]. Available: <http://arxiv.org/abs/2207.14131> (visited on 04/02/2025).
- [68] E. Kaufmann, L. Bauersfeld, and D. Scaramuzza, *A Benchmark Comparison of Learned Control Policies for Agile Quadrotor Flight*, arXiv:2202.10796 [cs], Feb. 2022. DOI: 10.48550/arXiv.2202.10796. [Online]. Available: <http://arxiv.org/abs/2202.10796> (visited on 05/16/2025).
- [69] P. Foehn, E. Kaufmann, A. Romero, *et al.*, “Agilicious: Open-Source and Open-Hardware Agile Quadrotor for Vision-Based Flight,” *Science Robotics*, vol. 7, no. 67, eabl6259, Jun. 2022, arXiv:2307.06100 [cs], ISSN: 2470-9476. DOI: 10.1126/scirobotics.abl6259. [Online]. Available: <http://arxiv.org/abs/2307.06100> (visited on 05/19/2025).
- [70] A. Paszke, S. Gross, S. Chintala, *et al.*, “Automatic differentiation in PyTorch,” en, Oct. 2017. [Online]. Available: <https://openreview.net/forum?id=BJJsrnfCZ> (visited on 05/25/2025).
- [71] A. Loquercio, E. Kaufmann, R. Ranftl, A. Dosovitskiy, V. Koltun, and D. Scaramuzza, “Deep Drone Racing: From Simulation to Reality With Domain Randomization,” *IEEE Transactions on Robotics*, vol. 36, no. 1, pp. 1–14, Feb. 2020, ISSN: 1941-0468.

- DOI: 10.1109/TR0.2019.2942989. [Online]. Available: <https://ieeexplore.ieee.org/document/8877728/> (visited on 05/26/2025).
- [72] Y. Song, M. Steinweg, E. Kaufmann, and D. Scaramuzza, *Autonomous Drone Racing with Deep Reinforcement Learning*, arXiv:2103.08624 [cs], Aug. 2021. DOI: 10.48550/arXiv.2103.08624. [Online]. Available: <http://arxiv.org/abs/2103.08624> (visited on 06/24/2025).
- [73] K. He, X. Zhang, S. Ren, and J. Sun, *Deep Residual Learning for Image Recognition*, arXiv:1512.03385 [cs], Dec. 2015. DOI: 10.48550/arXiv.1512.03385. [Online]. Available: <http://arxiv.org/abs/1512.03385> (visited on 06/15/2025).
- [74] R. Hartley and A. Zisserman, *Multiple View Geometry in Computer Vision*, 2nd ed. Cambridge: Cambridge University Press, 2004, ISBN: 978-0-521-54051-3. DOI: 10.1017/CB09780511811685. [Online]. Available: <https://www.cambridge.org/core/books/multiple-view-geometry-in-computer-vision/0B6F289C78B2B23F596CAA76D3D43F7A> (visited on 06/16/2025).
- [75] I. Geles, L. Bauersfeld, A. Romero, J. Xing, and D. Scaramuzza, *Demonstrating Agile Flight from Pixels without State Estimation*, arXiv:2406.12505 [cs], Jun. 2024. DOI: 10.48550/arXiv.2406.12505. [Online]. Available: <http://arxiv.org/abs/2406.12505> (visited on 06/28/2025).